



自动化测试

南京大学 软件学院 iSE实验室



课程目标



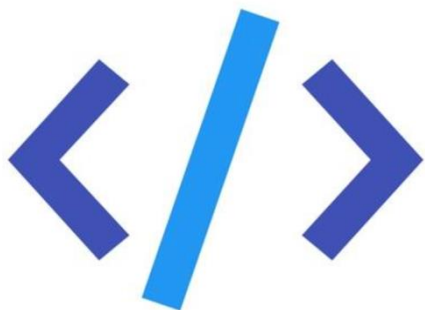
Integrating Software Test Automation



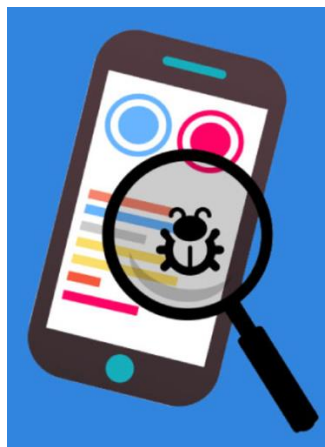
工业场景下的自动化、智能化测试技术



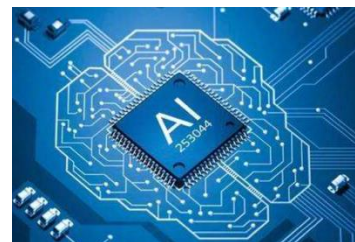
课程内容



源代码



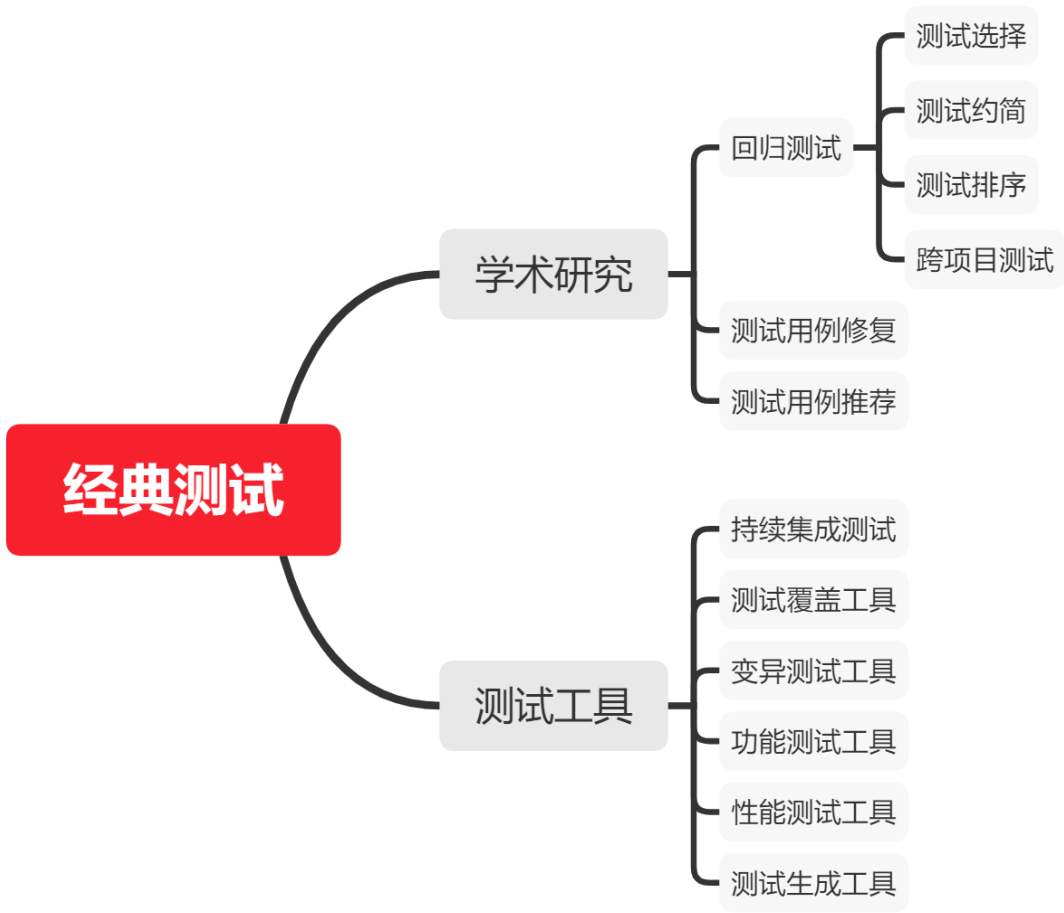
移动应用



智能软件



课程总览





课程总览





课程总览



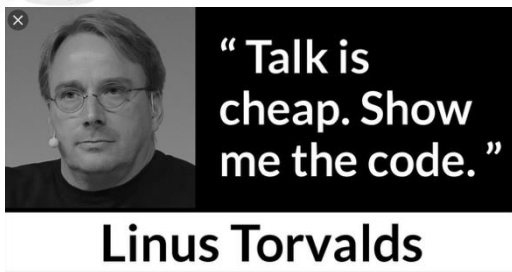


课程考核

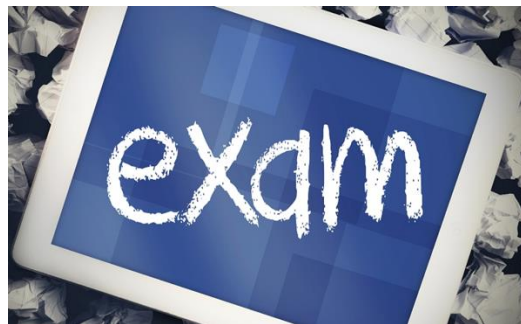


Literature Review

选一个方向**Survey**



选另外一个方向实现工具



还得期末考试



课程考核



课堂报告



课堂小测试

<https://www.icourse163.org/course/0809NJU015-1001773008>

[《软件测试》](#) [《自动化测试》](#)

<https://www.icourse163.org/course/20201248-1462096166>

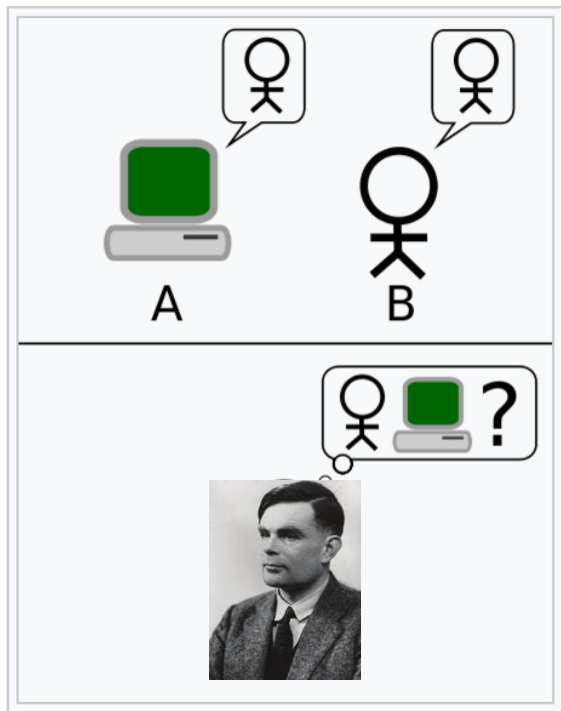


在线开放课程

课后小测试



图灵测试



假如你来提问，你会问什么问题？

假如你来回答，你会如何来作答？

假如你来判定，你会何时来终止？



测试的基本问题



测试设计问题

假如你来提问，你会问什么问题？

测试预言问题

假如你来回答，你会如何来作答？

测试终止问题

假如你来判定，你会何时来终止？

自动化是软件测试的一个梦想!





自动化软件测试的 系统性研究与实践



自动化测试生成

自动化测试执行

自动化测试优化

自动化测试修复

自动化测试扩增

自动化结果分析



 学术研究

单点技术突破后的工程应用

 面向大型企业

融入已有闭环产品链的工程应用

 面向中小企业

打造完整闭环产品链的工程应用

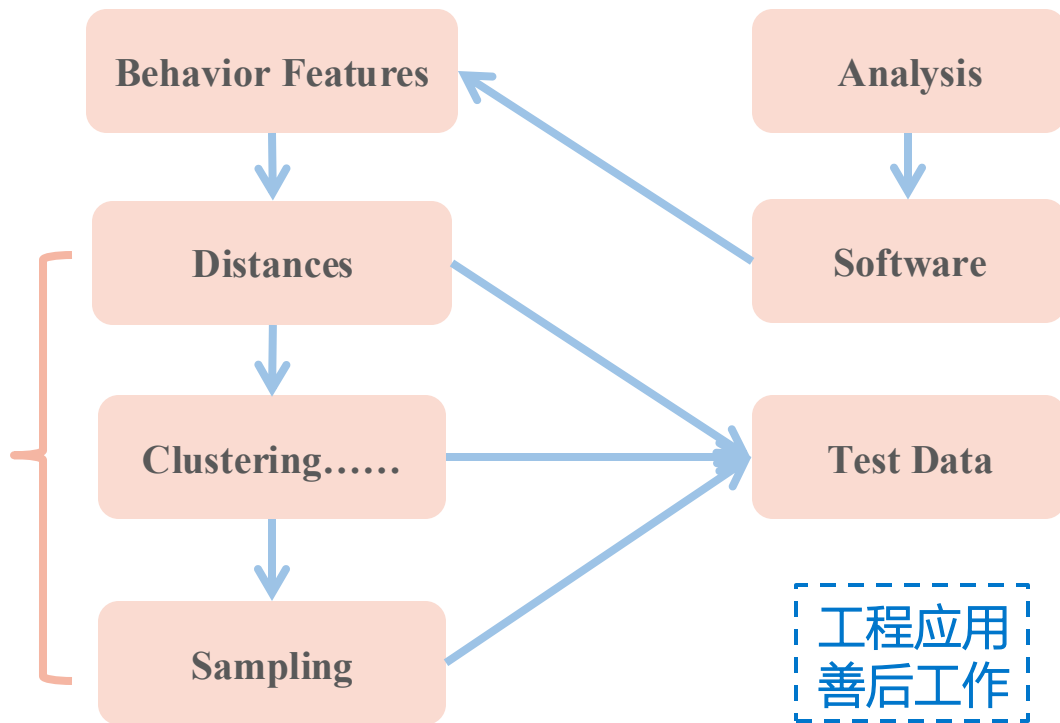


机器学习驱动 测试优化



工程重点

研究重点



- 执行路径比覆盖更具行为表征意义
- 执行路径同时复合了代码权重
- 抽样方式更具多样性和资源弹性



软件失效行为 刻画挑战



执行路径距离示例

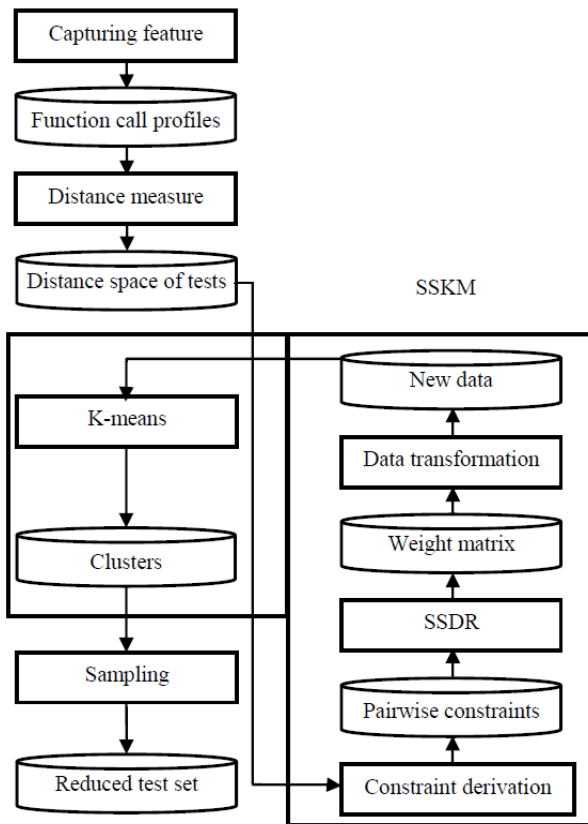
- 海明距离等
- $D(t1, t2) = 4$
- $D(t1, t3) = 3$
- $D(t1, t2) > D(t1, t3)$
- $[t1, t3]$ 比 $[t1, t2]$ 更相似

由于软件系统结构和缺陷产生机理的复杂性
执行路径并不能完全准确刻画软件失效行为

	t1	t2	t3	t4
s1	1	0	1	0
s2	1	0	0	1
s3	0	1	0	1
s4	1	1	1	0
s5	1	0	1	1
s6	0	0	1	1
s7	1	1	0	1



人机协同智能 人工标注+半监督



标注信息

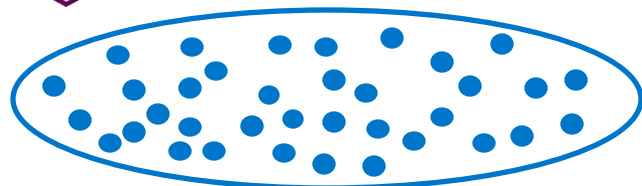
- Must-Link: 相同软件行为 M
- Cannot-Link: 不同软件行为 C

$$J(w) = \frac{1}{2n^2} \sum_{i,j} (w^T x_i - w^T x_j)^2$$

寻找行为空间转换矩阵 W
极大化优化目标 $J(W)$

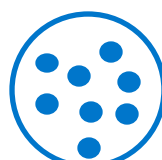
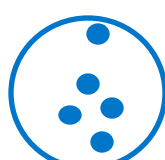
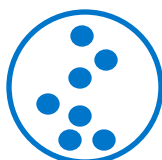
$$+ \frac{\alpha}{2n_C} \sum_{(x_i, x_j) \in C} (w^T x_i - w^T x_j)^2 \quad (3)$$
$$- \frac{\beta}{2n_M} \sum_{(x_i, x_j) \in M} (w^T x_i - w^T x_j)^2$$

Songyu Chen, **Zhenyu Chen**, Zhihong Zhao, Baowen Xu, Yang Feng. Using Semi-supervised Clustering to Improve Regression Test Selection Techniques ICST 2011



机器学习

软件分析



动态抽样
ICST 2010

半监督
ICST 2011

逻辑测试
SCIS 2012

多标签
ICSE 2012

动态加权
JSS 2014

相似性排序
SQJ 2014

动静态融合
ISSTA 2014

众包报告
FSE 2015





软件缺陷的触发与传播机理



什么叫Bug



- Software Fault : A **static** defect in the software (i.e., defect)
- Software Error : An incorrect **internal** state that is the manifestation of some fault
- Software Failure : **External**, incorrect behavior with respect to the requirements or other description of the expected behavior



Fault



- Software Fault : A static defect in the software (i.e., defect)

```
public static void CSta (int [ ] numbers)
{
    int length = numbers.length;
    double mean, sum;

    sum = 0.0;
    for (int i = 1; i < length; i++)//i=0
    {
        sum += numbers [ i ];
    }
    mean = sum / (double) length;

    System.out.println ("mean: " +
    mean);
}
```





Error



- Software Error : An incorrect internal state that is the manifestation of some fault

```
public static void CSta (int [ ] numbers)
{
    int length = numbers.length;
    double mean, sum;

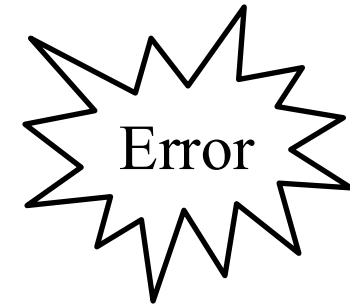
    sum = 0.0;
    for (int i = 1; i < length; i++)//i=0
    {
        sum += numbers [ i ];
    }
    mean = sum / (double) length;

    System.out.println ("mean: " +
mean);
}
```

Test Input: [3,4,5]

sum=3+4+5

sum=4+5





Failure



- Software Failure : External, incorrect behavior with respect to the requirements or other description of the expected behavior

```
public static void CSta (int [ ] numbers)
{
    int length = numbers.length;
    double mean, sum;

    sum = 0.0;
    for (int i = 1; i < length; i++)//i=0
    {
        sum += numbers [ i ];
    }
    mean = sum / (double) length;

    System.out.println ("mean: " +
mean);
}
```

Test Input: [3,4,5]

sum=3+4+5

sum=4+5

mean=4

mean=3





PIE Model



1. Execution: The location or locations in the program that contain the fault must be reached
2. Infection : The state of the program must be incorrect
3. Propagation : The infected state must propagate to cause some output of the program to be incorrect



A test executing the fault may not produce an error!

```
public static void CSta (int [ ] numbers)
{
    int length = numbers.length;
    double mean, sum;

    sum = 0.0;
    for (int i = 1; i < length; i++)//i=0
    {
        sum += numbers [ i ];
    }
    mean = sum / (double) length;

    System.out.println ("mean: " +
    mean);
}
```

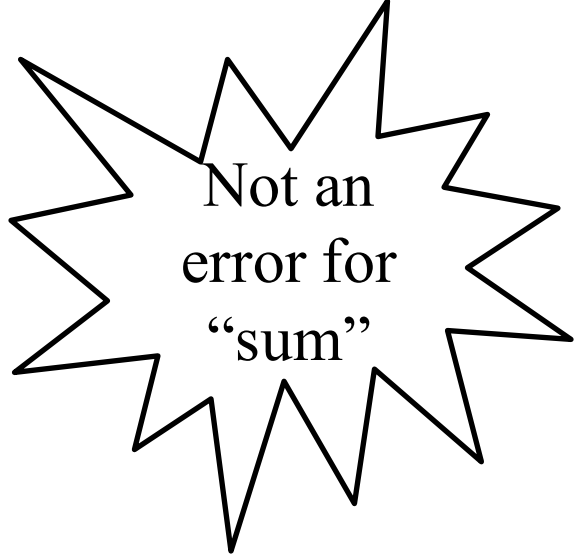
Test Input: [0,4,5]

sum=0+4+5

sum=4+5

mean=3

mean=3



Not an
error for
“sum”



An error may not be propagated to the output!

```
public static void CSta (int [ ] numbers)
{
    int length = numbers.length-1;
    double mean, sum;

    sum = 0.0;
    for (int i = 0; i < length; i++)
    {
        sum += numbers [ i ];
    }
    mean = sum / (double) length;

    System.out.println ("mean: " +
    mean);
}
```

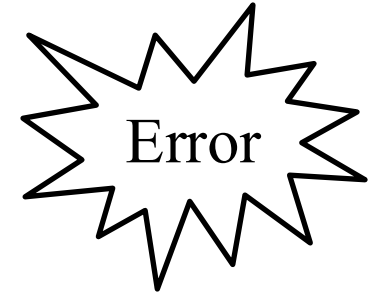
Test Input: [3,5,4]

sum=3+5+4

sum=3+5

mean=12/3=4

mean=8/2=4

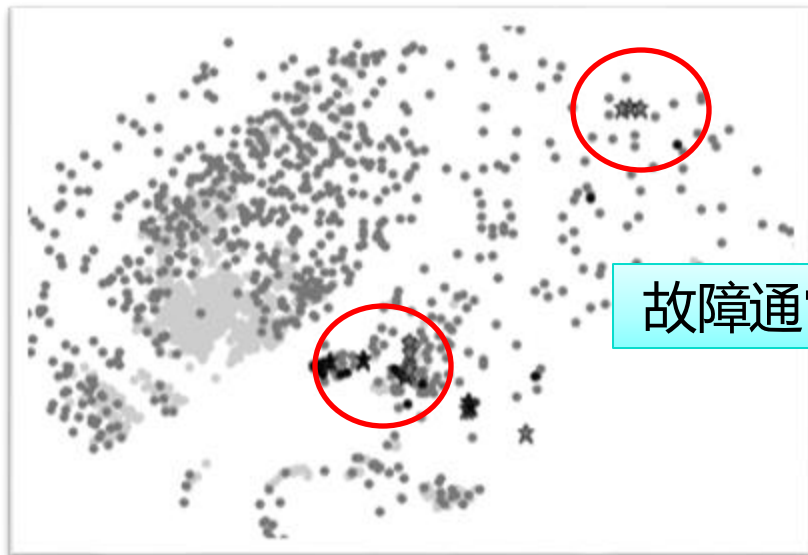




Bug空间聚集



William Dickinson et al. @ FSE, 2001.



MDS Display of Distribution of Failures in Profile Space on LilyPond

William Dickinson et al. @ FSE, 2001.



MDS Display of Distribution of Failures in Profile Space on GCC

故障通常聚在一起



测试的局限性



三大挑战

输入空间庞大

无法穷举所有输入

实现逻辑复杂

无法想到所有场景

测试预言未知

无法判定测试的预期输出



随机测试



大数定律

测试执行次数够多

测试数据随机生成

概率低的偶然现象发生



发现的Bug类型



断言失败

01

异常崩溃

02

03

无效输入

04

错误输出

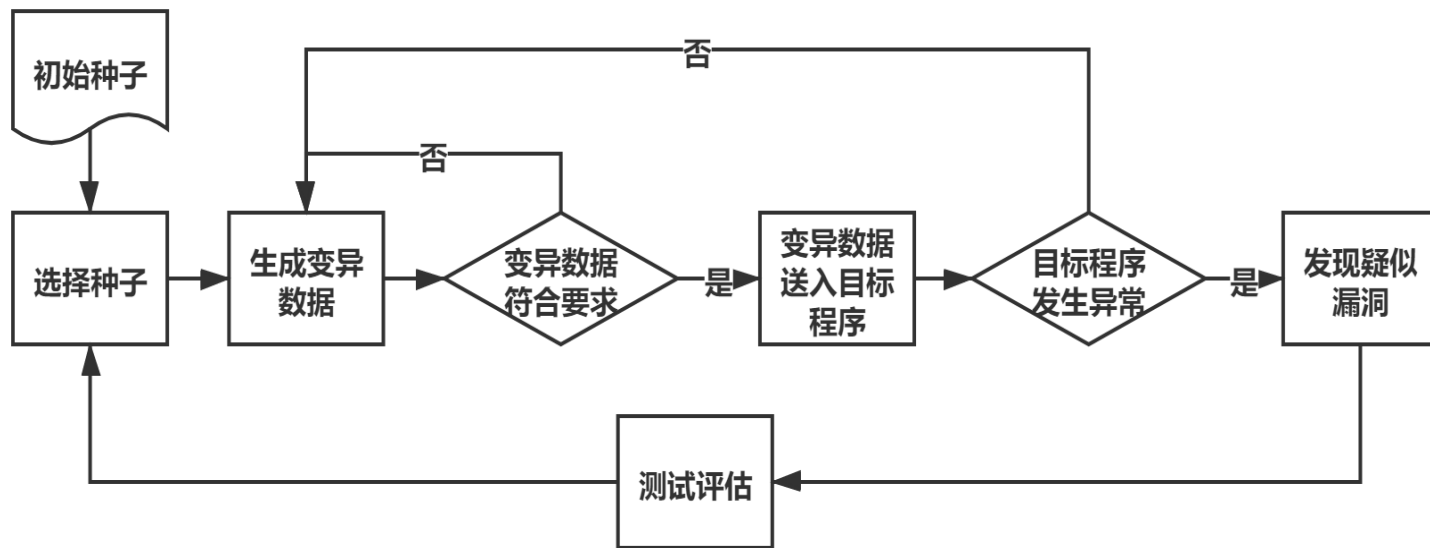




模糊测试



模糊测试(Fuzzing)是一项通过异常的输入自动化发现待测程序缺陷的技术。





经典测试技术



变异测试

蜕变测试

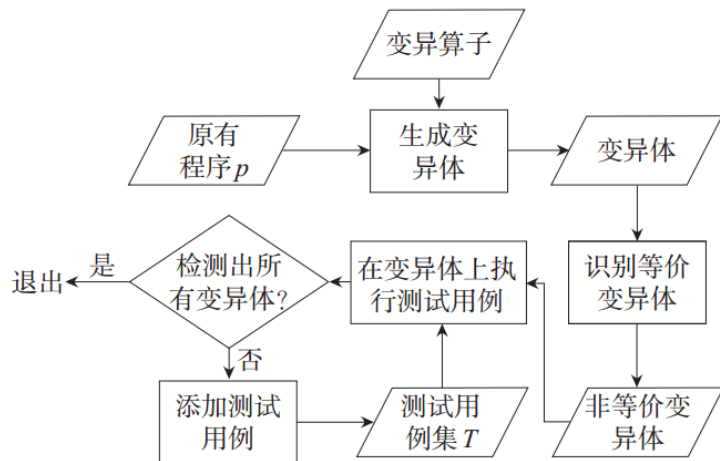
差分测试



变异测试



Program p	Equivalent Mutant m
<pre>for (int i = 0; i < 10; i++) { ...(the value of i is not changed) }</pre>	<pre>for (int i = 0; i != 10; i++) { ...(the value of i is not changed) }</pre>



变异测试旨在找出有效的测试用例，发现程序中真正的错误。

R. Lipton, "Fault Diagnosis of Computer Programs", Student Report, CMU, 1971.

测试输入如何生成



MR6: $\sin^2(x) + \sin^2(\pi/2 - x) = 1$

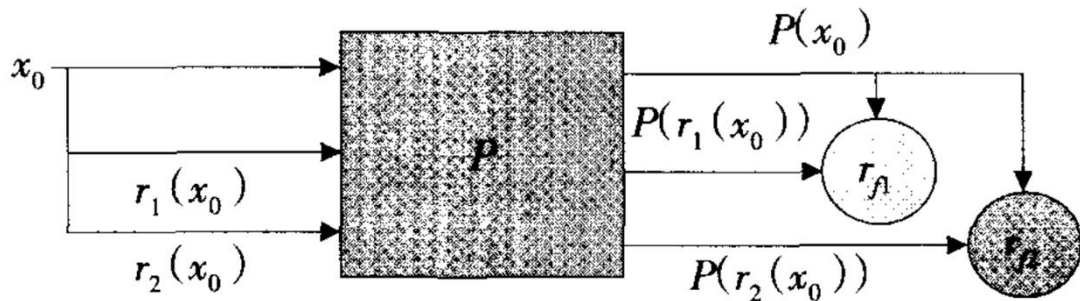


Fig.1 Illustration of metamorphic testing

蜕变关系(Metamorphic Relation, MR) 是指多次执行目标程序时, 输入与输出之间期望遵循的关系。

T.Y. Chen, S.C. Cheung, S.M. Yiu, Metamorphic Testing: A New Approach for Generating Next Test Cases, HKUST-CS98-01, 1998

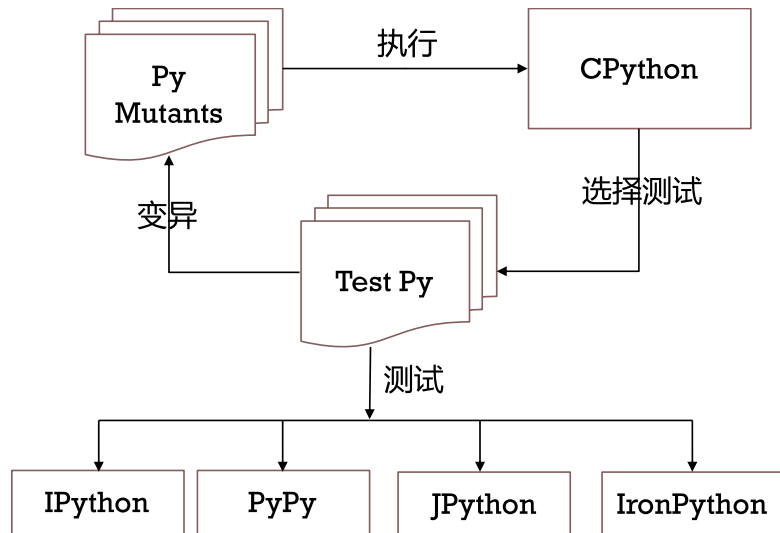
测试预言如何获取



差分测试



基本思想：通过将同一测试用例运行到一系列相似功能的应用中观察执行差异来检测bug。

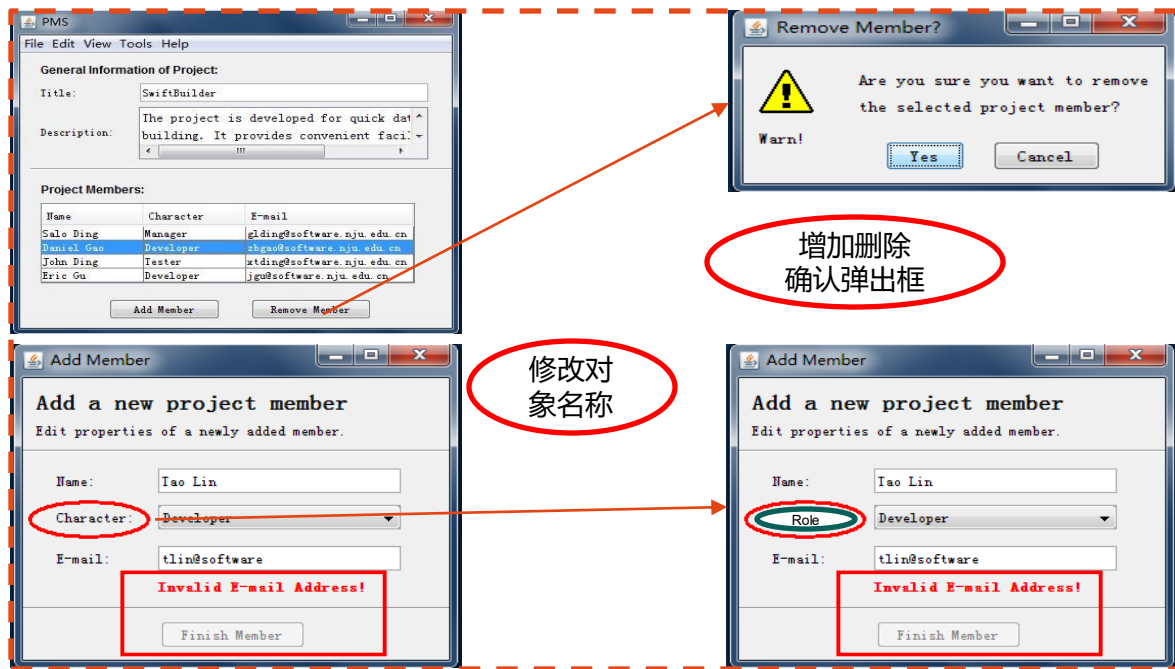


William M. McKeeman, "Differential testing for software," Digital Technical Journal, vol. 10, no. 1, pp. 100–107, 1998.

测试结果如何判断



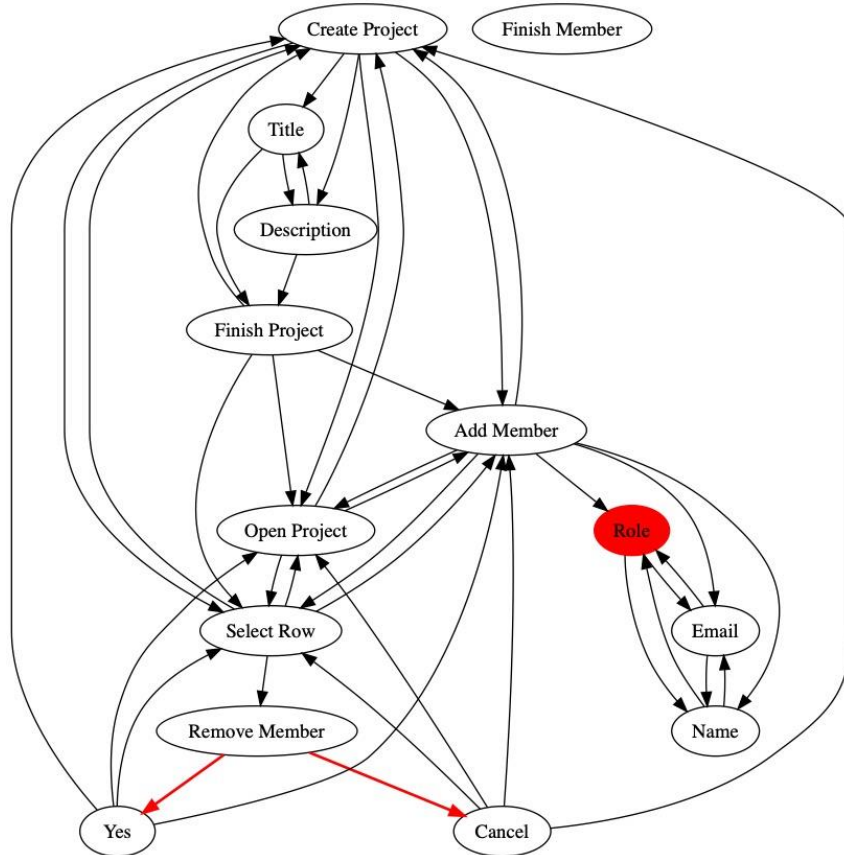
自动化测试脚本修复



- 人工测试脚本内
含业务理解信息
- 界面变更给GUI
测试脚本维护带
来巨大挑战
- 快速迭代需要自
动化测试脚本



Event Flow Graph



TS_1

- 1 Window("PMS").Menu("File").Menu("Create Project").Select
- 2 Window("PMS").Dialog("Create Project").Field("Title").Set "projTest"
- 3 Window("PMS").Dialog("Create Project").Field("Description").Set "To test creating a project"
- 4 Window("PMS").Dialog("Create Project").Button("Finish Project").Click

TS_2

- 1 Window("PMS").Menu("File").Menu("Open Project").Select
- 2 Window("PMS").Button("Add Member").Click
- 3 Window("PMS").Dialog("Add Member").Field("Name").Set "Tao Lin"
- 4 Window("PMS").Dialog("Add Member").Field("Character").Set "Developer"
- 5 Window("PMS").Dialog("Add Member").Field("E-mail").Set "tlin@software.nju.edu.cn"
- 6 Window("PMS").Dialog("Add Member").Button("Finish Member").Click

TS_3

- 1 Window("PMS").Menu("File").Menu("Open Project").Select
- 2 Window("PMS").Table("Project Members").SelectRow #2
- 3 Window("PMS").Button("Remove Member").Click
- 4 Window("PMS").Table("Project Members").SelectRow #1

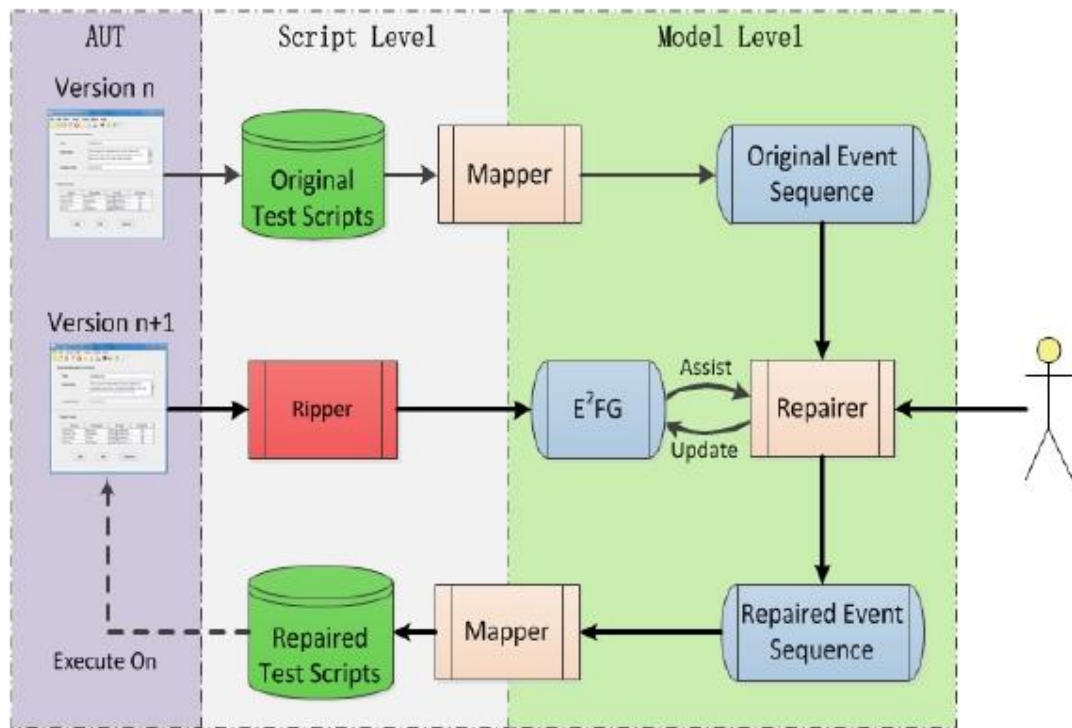
TS_4

- 1 Window("PMS").Menu("File").Menu("Open Project").Select
- cp1 Window("PMS").Table("Project Members").CheckProperty("size", 4)
- 2 Window("PMS").Table("Project Members").SelectRow #1
- 3 Window("PMS").Button("Remove Member").Click
- 4 Window("PMS").Table("Project Members").SelectRow #1
- 5 Window("PMS").Button("Add Member").Click
- 6 Window("PMS").Dialog("Add Member").Field("Character").Set "Tester"
- 7 Window("PMS").Dialog("Add Member").Field("Name").Set "John Ding"
- 8 Window("PMS").Dialog("Add Member").Field("E-mail").Set "johnding@cs.umd.edu"
- cp2 Window("PMS").Table("Project Members").CheckProperty("size", 3)
- 9 Window("PMS").Dialog("Add Member").Button("Finish Member").Click
- cp3 Window("PMS").Table("Project Members").CheckProperty("size", 4)

TS_5

- 1 Window("PMS").Menu("File").Menu("Create Project").Select
- 2 Window("PMS").Dialog("Create Project").Field("Title").Set "newProject"
- 3 Window("PMS").Dialog("Create Project").Button("Finish Project").Click
- cp1 Window("PMS").Field("Name").CheckProperty("text", "newProject")
- 4 Window("PMS").Button("Add Member").Click
- 5 Window("PMS").Dialog("Add Member").Field("Name").Set "Eric Gu"
- 6 Window("PMS").Dialog("Add Member").Field("Character").Set "Developer"
- 7 Window("PMS").Dialog("Add Member").Field("E-mail").Set "ericgu@cs.umd.edu"
- cp2 Window("PMS").Dialog("Add Member").Field("Character").CheckProperty("value", "Developer")
- 8 Window("PMS").Dialog("Add Member").Button("Finish Member").Click
- cp3 Window("PMS").Table("Project Members").CheckProperty("size", 1)

人机协同智能： 人工辅助自动化修复



- 受限于当前软件分析技术, EFG元素可能不完整
- 修复流程卡壳时给测试人员选择权
- 系统记录人工信息形成知识库加速后期修复



自动化测试脚本修复

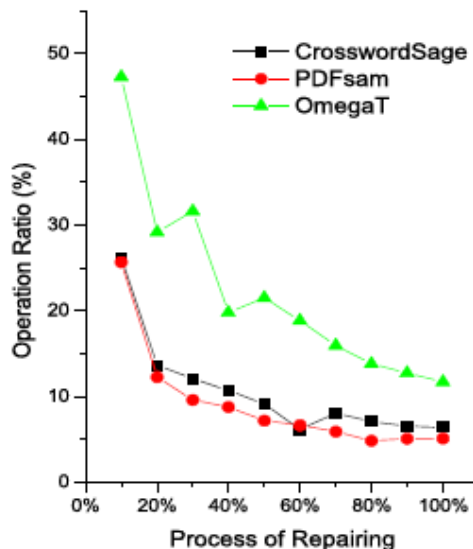


Table 6: Repairing Results

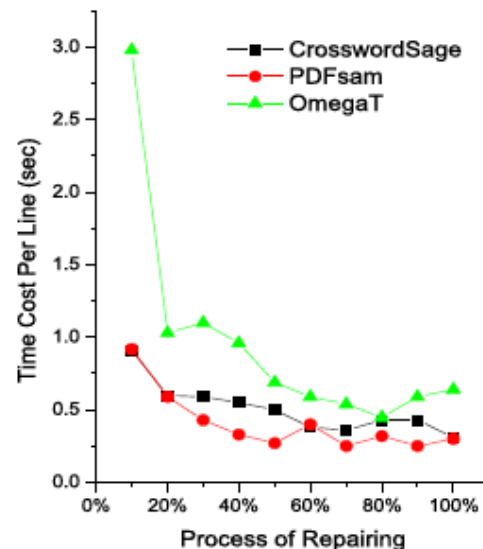
Application	Unusable <i>TSs</i> #	Repaired <i>TSs</i> #	Ratio
Crossword Sage	88	45	51%
PDFsam	148	133	90%
OmegaT	129	106	82%

Table 7: Time Cost Effectiveness (minutes)

Application	Repairing	Creation	Ratio
CrosswordSage	29.4	1317	2.2%
PDFsam	29.0	1342	2.2%
OmegaT	71.7	1415	5.1%



(a) Manual Operation Cost



(b) Time Cost Per Line

2-5%的人工参与，51%-90%的修复率，知识积累使得人工工作量快速下降



程序修复



01 规约提取阶段

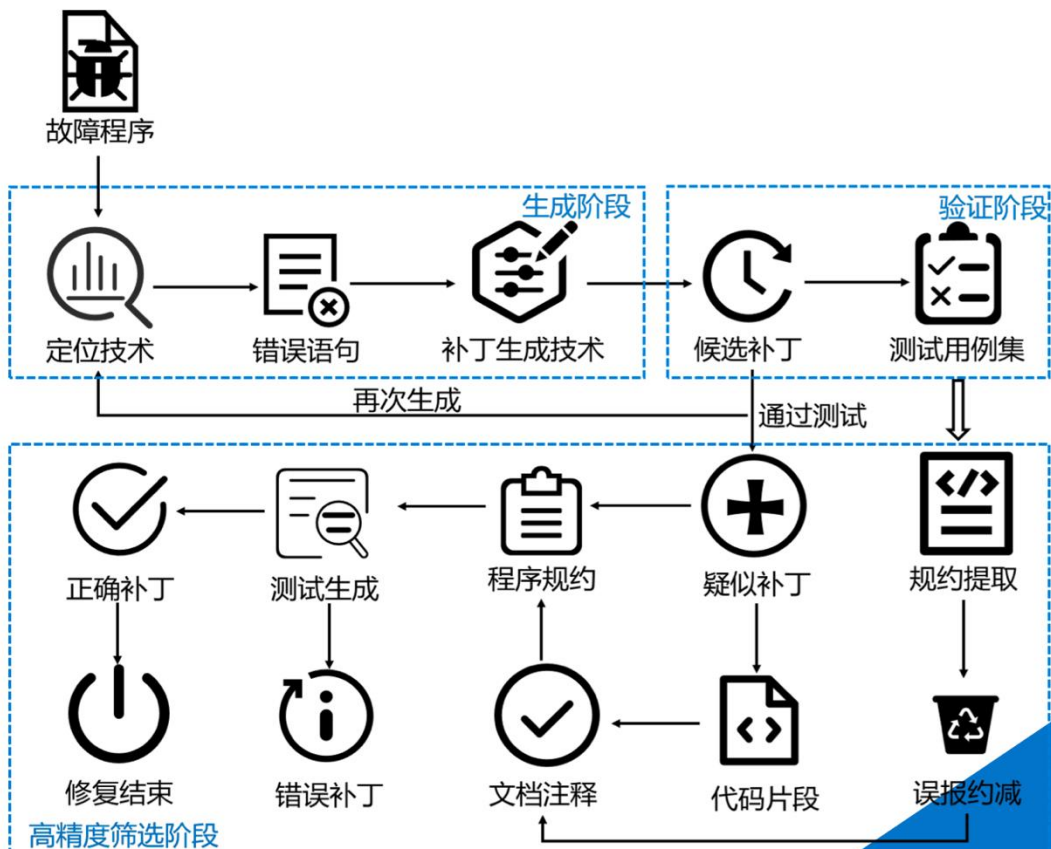
1. 对每个故障代码片段分析注释文档信息。
2. 使用自然语言处理技术提取其程序规约。

02 误报约减阶段

1. 通过原程序已有测试用例集，对每个故障代码片段分析提取其正确的程序规约。
2. 通过冲突依赖分析对自动生成的程序规约进行误报约减，获得高准确率的规约。

03 切片筛选阶段

1. 对原有测试用例集进行代码切片分析，提取其最小触发故障的测试用例片段。
2. 根据约减后的规约进行测试用例扩增，使用扩增用例此进一步筛选冗余补丁。





测试用例推荐

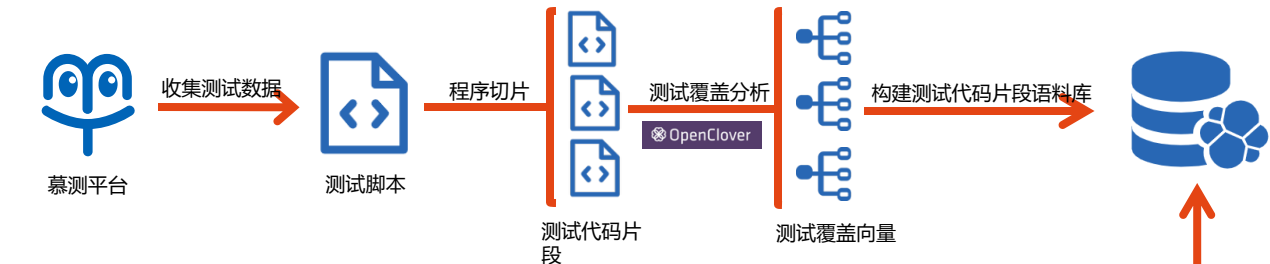


01 离线数据处理流程

程序切片: Java Parser & Wala

覆盖分析: OpenClover

语料存储: Elasticsearch

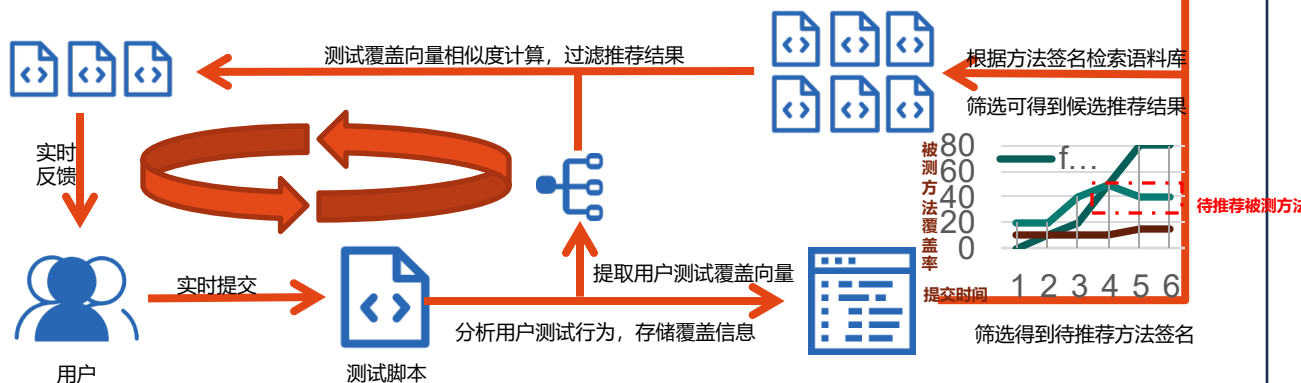


02 动态代码推荐流程

覆盖分析: OpenClover

相似分析: Jaccard

实时反馈: Event机制 WebSocket



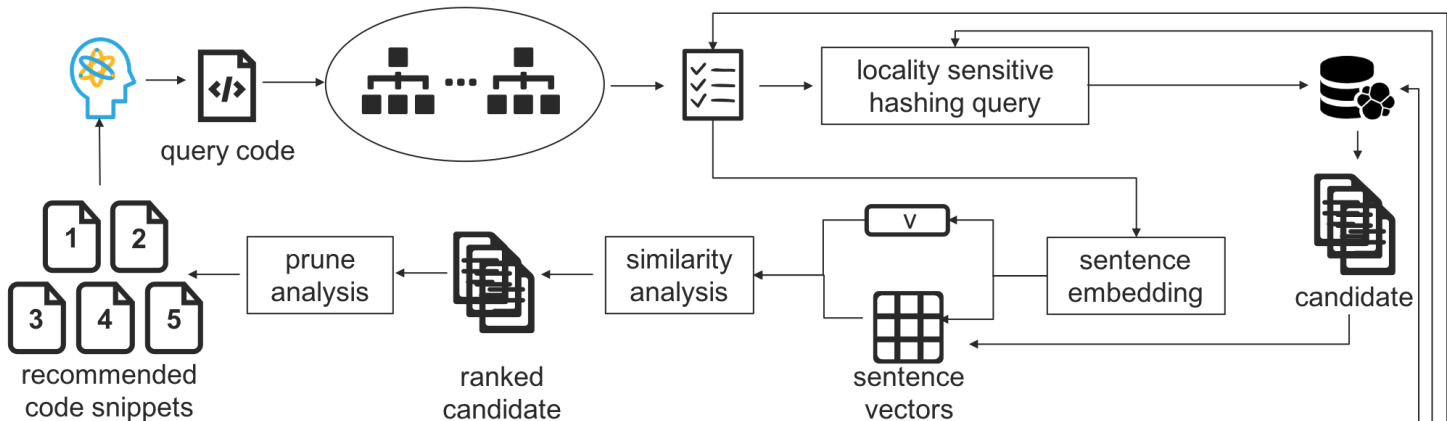


测试用例推荐



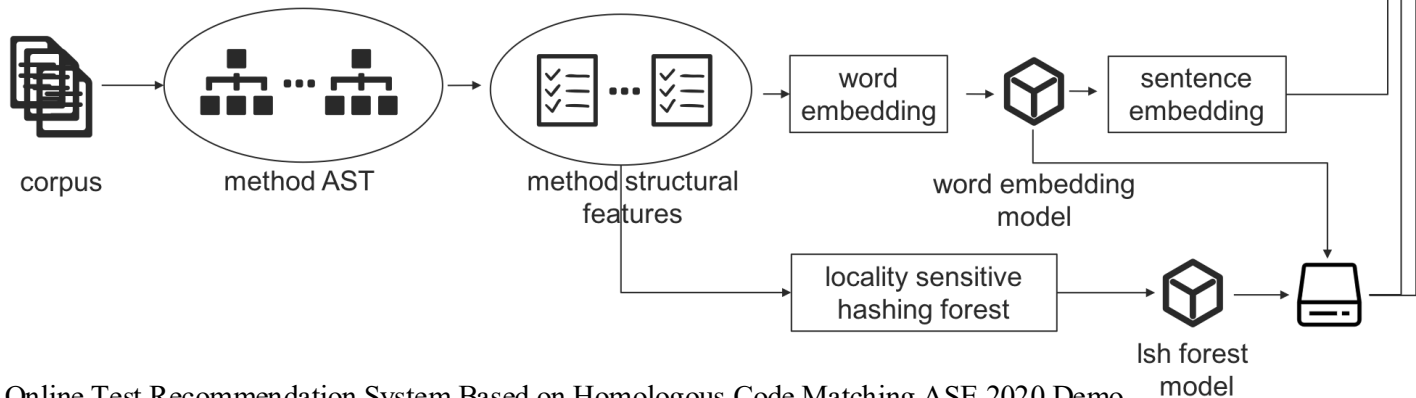
01 结构特征提取

语料来源: Github
解析器: Antlr4
近邻过滤: LSH



02 结构特征分析

词嵌入: FastText
句嵌入: SIF
相似分析: Cosine
语料存储: Elasticsearch





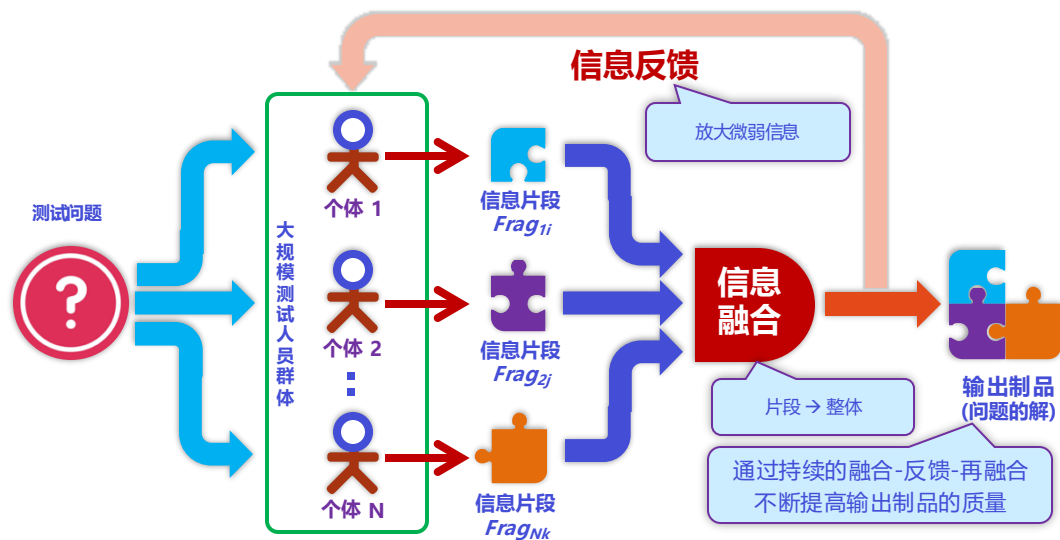
产教研融合测试 (2013)



- P1 百度安卓输入法
✓ 功能+兼容性
- P2 百度浏览器
✓ 功能+易用性
- P3 百度影音播放器
✓ 易用+性能

- Quasi-Crowdsourcing Testing for Educational Projects. ICSE 2014-SEET
- Test Report Prioritization to Assist Crowdsourced Testing. ESEC/FSE 2015
- Guiding the Crowds for Android Testing, ICSE 2016-Poster
- Revisit of automatic debugging via human focus-tracking analysis ICSE 2016
- Multi-Objective Test Report Prioritization Using Image Understanding, ASE 2016
- ...

基于互联网群体 智能的软件测试





智能软件测试 示例-途牛



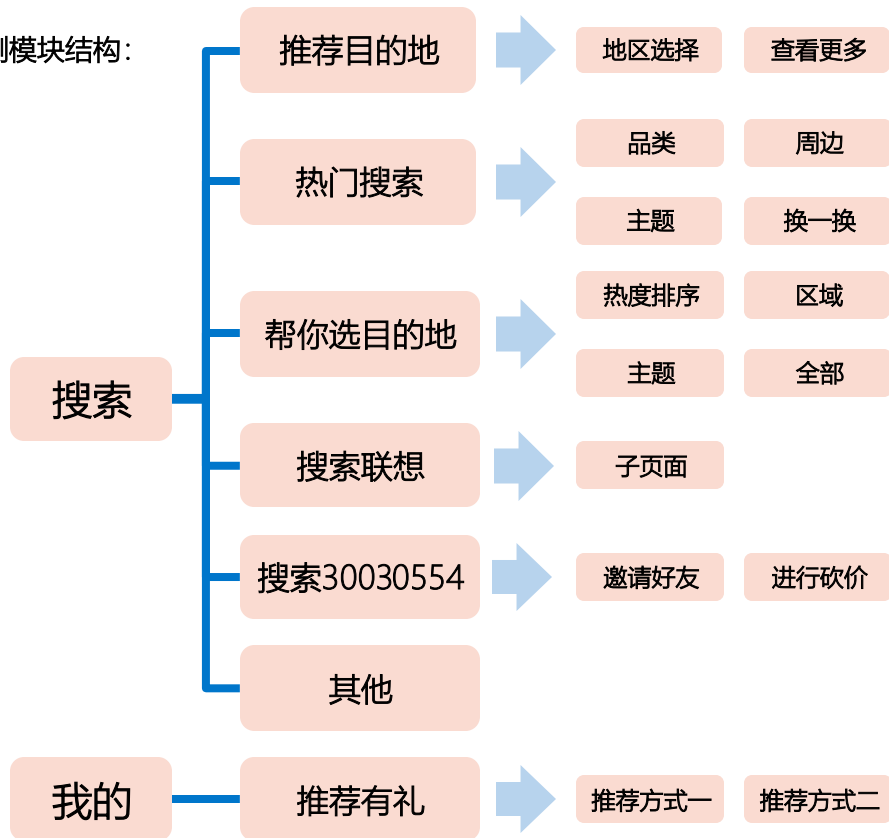
待测界面:



待测APP



待测模块结构:





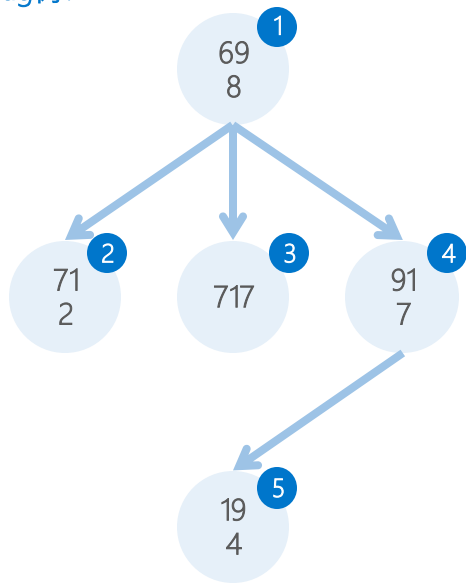
众包协作： 一棵Bug报告树的生长



相关Bug报告列表：

报告人	创建时间	Bug描述	截图	增量
吴**	8:05	点击查看更多目的地时候会app停止运行	停止运行截图	根节点，发现Bug
张**	8:07	1.在搜索框点击查看更多 2.进入查看更多界面，app停止运行	停止运行截图	描述步骤化
杨1*	8:09	当启动app第一次点击查看更多时， app会出现崩溃，之后重启错误不再发生	正常进入截图	只有第一次app会崩溃； 提供正常进入页面的截图
杨2*	9:03	1.推荐目的地查看更多时，可能造成app 停止运行或崩溃 2.推荐目的地查看更多时，可能造成app 需要加载时间较长	停止运行截图	步骤化 app需要较长加载时间
李*	9:46	打开应用进入搜索页面，点击推荐目的地 的查看更多，一定出现以下情况的一种： 1.app崩溃，停止运行 2.页面卡顿，长时间空白后才进入到正确的 查看更多页面	停止运行截图 页面卡顿截图	完备总结，页面卡顿截图

对应Bug树：



代表对应序号报告
代表提交时间序号



Bug报告协作过程

比赛时间：2018年11月16日

累计为 4 小时（8时-12时）

由用户提交的Bug报告生成Bug树

- 每个点表示一个Bug报告
- 点上的数字代表Bug的id
- 每十分钟生成一次

视频展示 ➡



用户协作过程



比赛时间：2018年11月16日

累计为 4 小时 (8时-12时)

用户之间产生Bug报告相互Fork

- 每个点表示一个用户
- 点的大小正比于提交Bug数目
- 括号内为该用户最后的得分。

视频展示





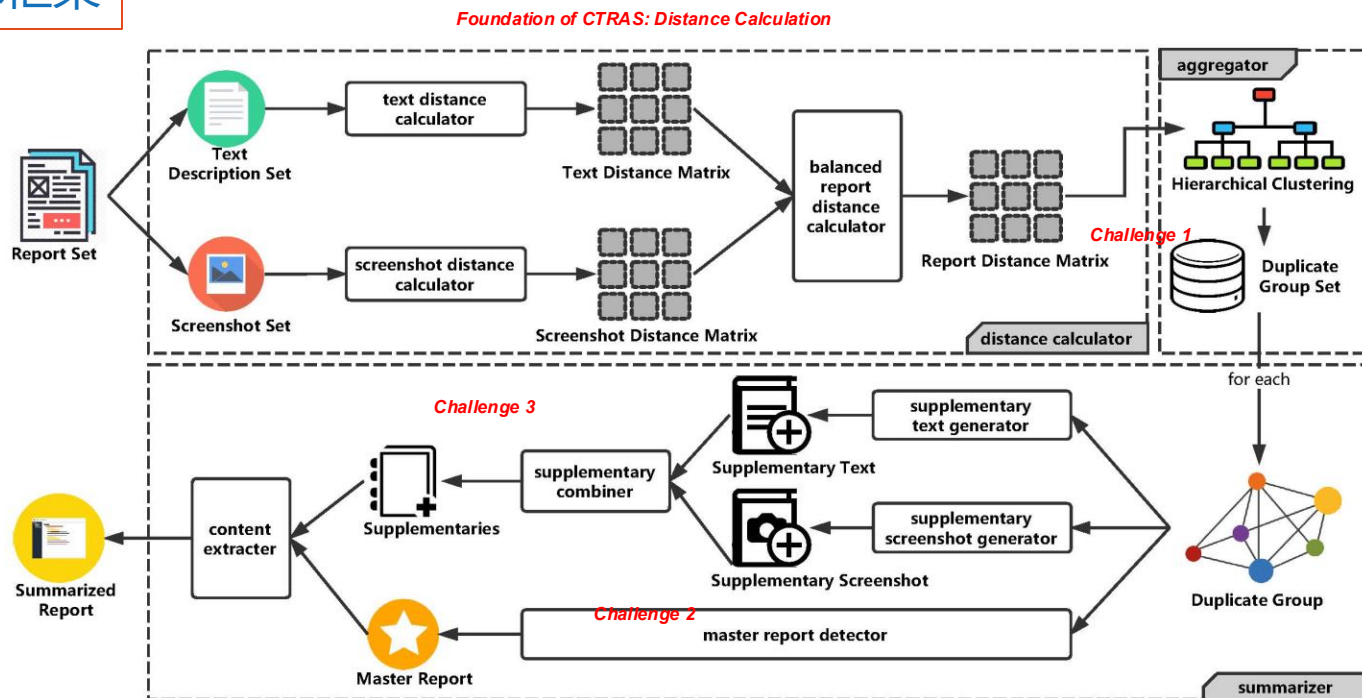
智能化的软件工程方法 人机协同+人人协同



面向群体智能的测试报告 自动化分析



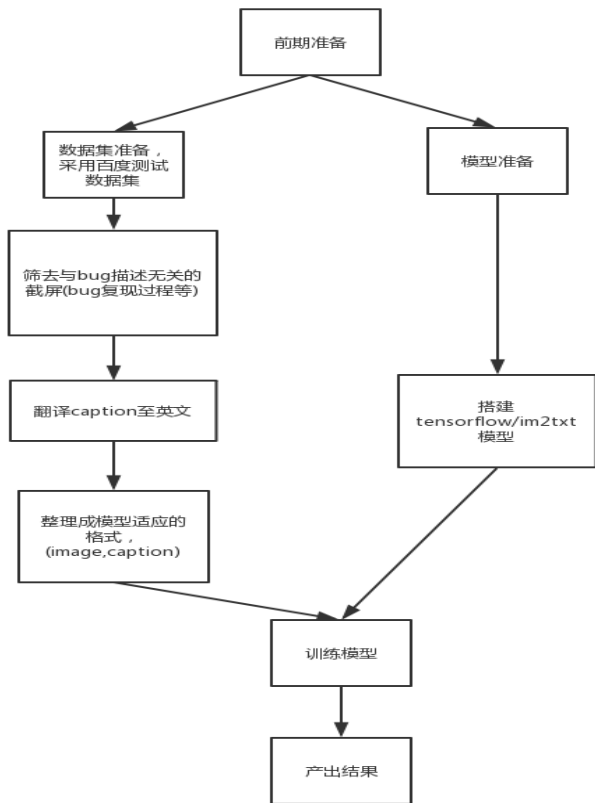
技术框架



Aggregating and Summarizing Crowdsourced Test Reports, submit to ICSE 2019



面向群体智能的Bug 截屏文本自动生成



01

选定模型：采用经典的 tensorflow/im2txt 模型进行实验

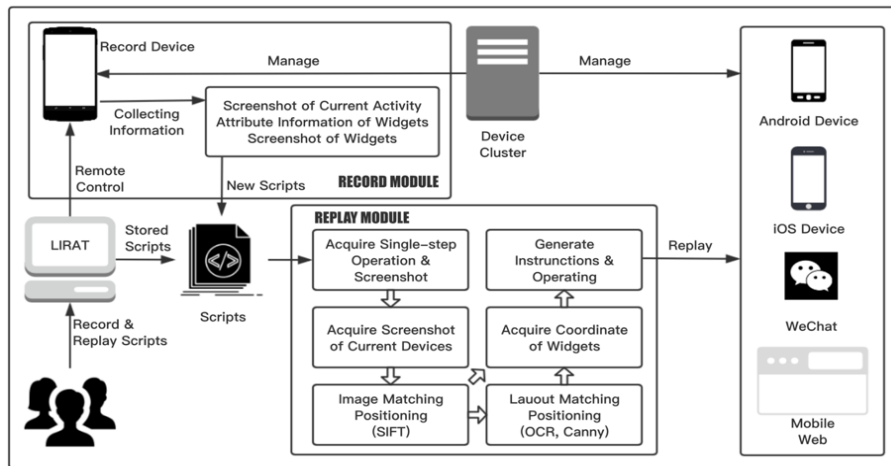
02

数据集：采用工业界测试数据集 总共包含35480个image-caption对

03

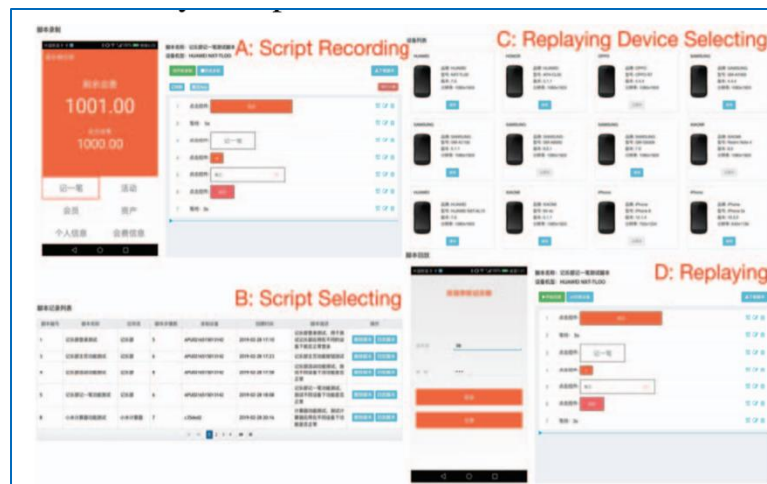
50w次迭代迭代训练

基于截图理解的 测试回放技术

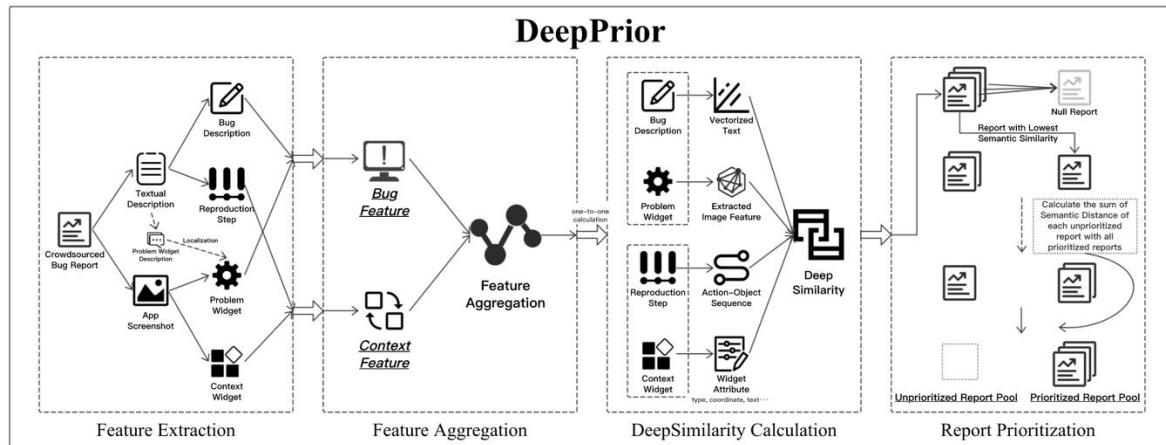


- 碎片化问题，加大测试难度
- 动态内容，相似控件难识别
- 基于图像理解技术，录制和回放跨平台的测试脚本

LIRAT: Layout and image recognition driving automated mobile testing of cross-platform. ASE 2019-Demo
Layout and Image Recognition Driving Cross-Platform Automated Mobile Testing. ICSE 2021



基于深度图像理解的报告排序



- 众测报告数量繁多，报告质量参差不齐
- 应用迭代频繁快速，报告审查需全覆盖
- 图像文本构成报告，相互映证缺陷内容
- 深度挖掘图像语义，融合文本有效排序

Algorithm 1 Crowdsourced Test Report Prioritization

Input: Crowdsourced Test Report Set $R_{initial}$

Output: Prioritized Crowdsourced Test Report Set P

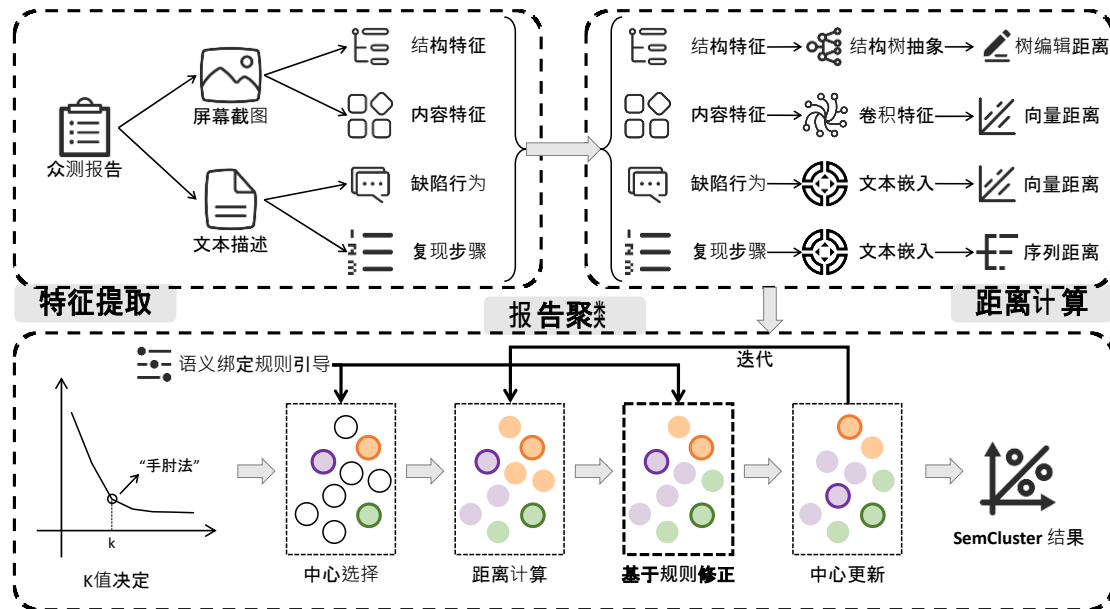
```

1: initiate unprioritized report pool  $U \leftarrow R_{initial}$ 
2: initiate prioritized report pool  $P = \emptyset$ 
3: initiate target report  $r_t$ 
4: initiate NULL Report  $r_{null}$ 
5:  $P.append(r_{null})$ 
6: while  $|U| \neq 0$  do
7:   initiate  $similarity = 2$ 
8:   for each  $r \in U$  do
9:     for each  $r_p \in P$  do
10:      initiate  $similarity_r = 2$ 
11:       $Sim_{BFT} = \alpha * Sim_{WP} + (1 - \alpha) * Sim_P$ 
12:       $Sim_{CFT} = \beta * Sim_{WC} + (1 - \beta) * Sim_R$ 
13:       $calSemSim() = \gamma * Sim_{BFT} + (1 - \gamma) * Sim_{CFT}$ 
14:       $similarity_r = calSemSim(r, r_p)$ 
15:      if  $calSemSim(r, r_p) < similarity_r$  then
16:         $similarity_r = calSemSim(r, r_p)$ 
17:      end if
18:    end for
19:    if  $similarity_r < similarity$  then
20:       $r_t = r$ 
21:       $similarity = similarity_r$ 
22:    end if
23:  end for
24:   $P.append(r_t)$ 
25:   $U.remove(r_t)$ 
26: end while
27: return  $P$ 

```



基于多源信息语义关联的众测报告半监督聚类



泛在场景海量众测报告

基于人类理解视角特征提取

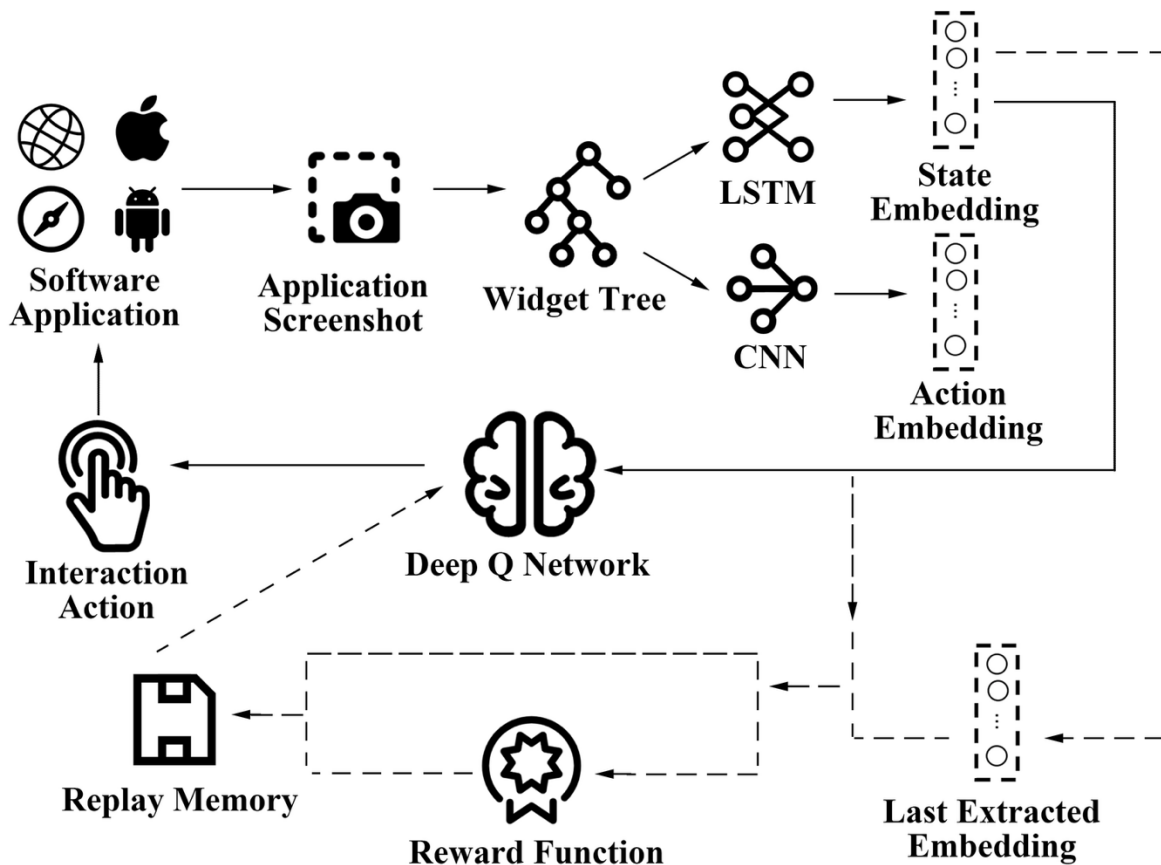
富语义图文绑定规则构建

聚类修正引导加强

众测报告优化，群体智能汇聚，提升开源社区众测参与者贡献度



基于强化学习和图像理解的 跨平台测试技术





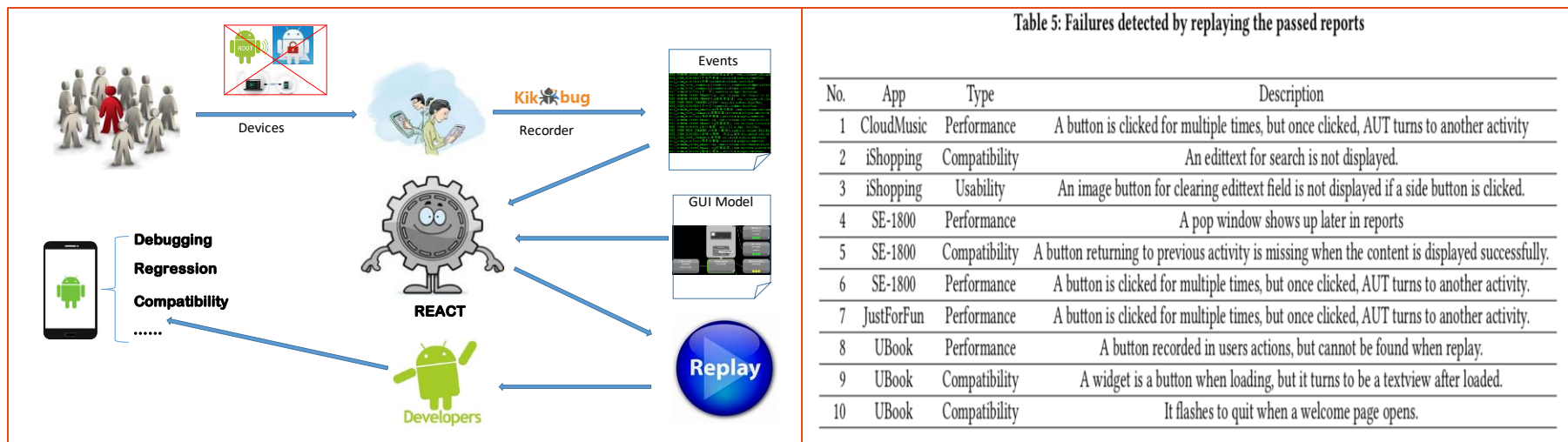
RQ1: (有效性) 此技术是否有效提高众包测试的质量?

RQ2: (效率) 此技术是否提高众包测试的效率?

RQ3: (覆盖率) 此技术是否提高众包测试的覆盖率?

RQ4: (可用性) 此技术对众包工人是否有用?





基于日志的轻量级测试自动回放

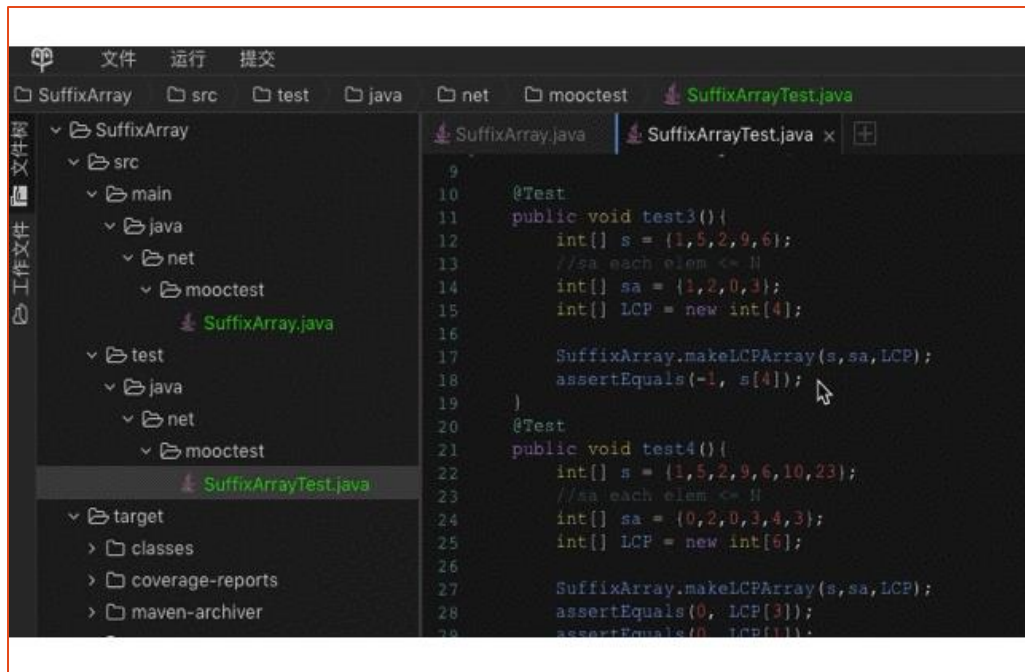


软件测试人机大战



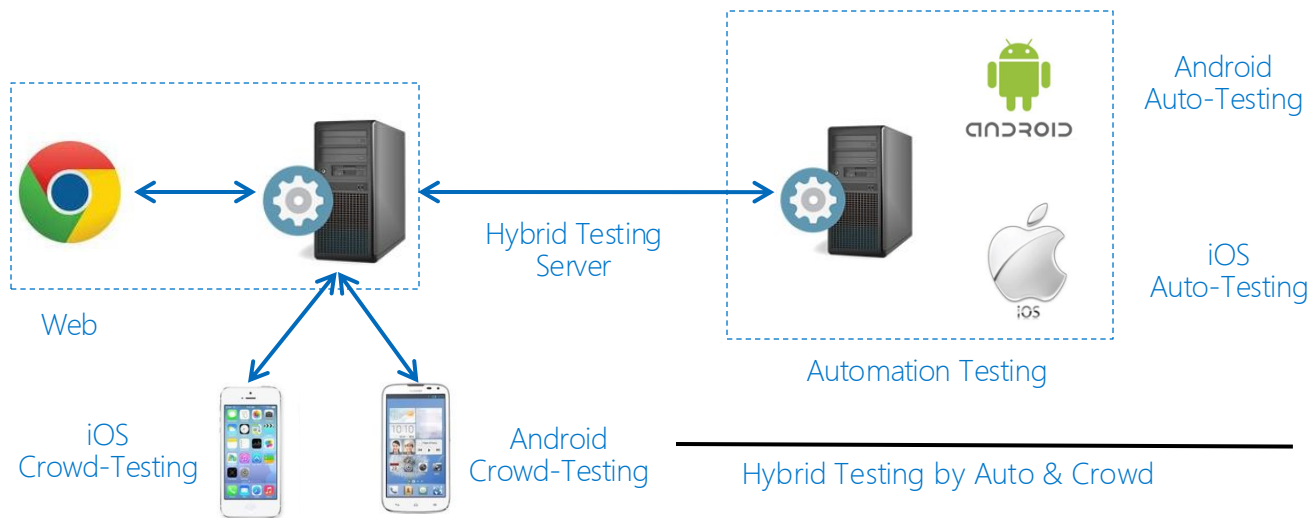
耗时 1 小时 38 分
范治宇首次超过所有机器

WebIDE的普及，便于
人机协同和人人协同编码





面向移动应用的 人机协同智能化测试



Hybrid Testing using Web&Apps



自动化软件测试 发展路线



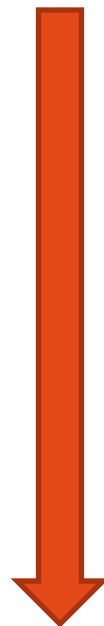
确定步骤的自动化



非确定步骤的自动化



具有学习能力的自动化



工程
应用

测试
大赛



智能软件的质量问题



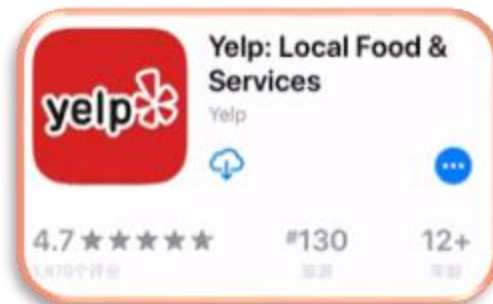
智能软件系统具有广泛应用前景



近年Amazon Alexa产品系列屡被爆出误报警深夜怪笑缺陷，对话场景复杂极端数据难以覆盖

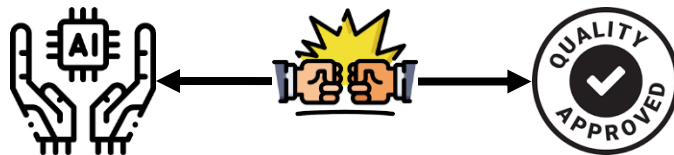


2018年Uber的自动驾驶测试车在进行路试时发生事故导致行人死亡，自动驾驶系统在不确定性开放环境难以充分测试



2019年Yelp基于深度神经网络的自动化缺陷修复助手删除了所有代码，应用场景语义难以充分考虑

- 任务目标难以精确表达
- 非确定性开放环境运行
- 应用于复杂软硬件场景



智能软件系统快速应用和质量保障手段相对滞后的矛盾

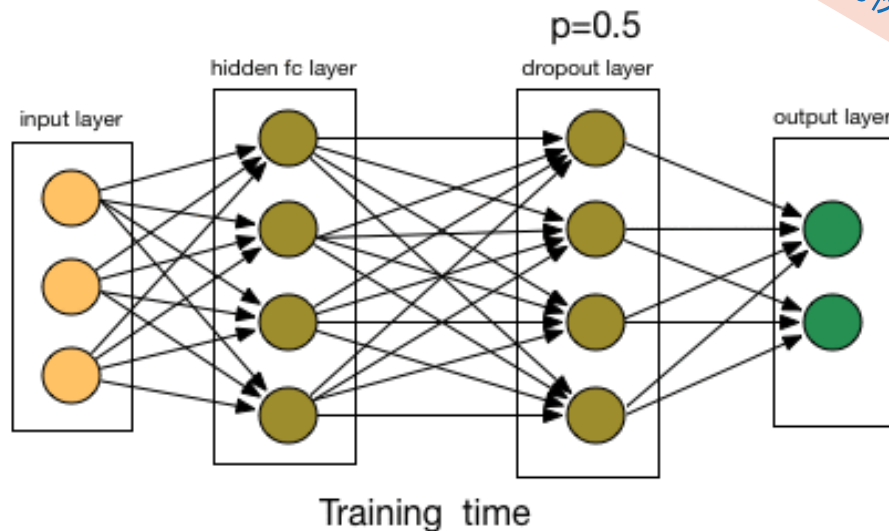
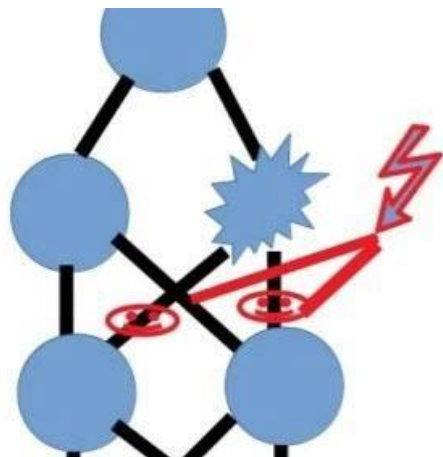


智能软件的Bug 是什么？



以深度学习
系统为例

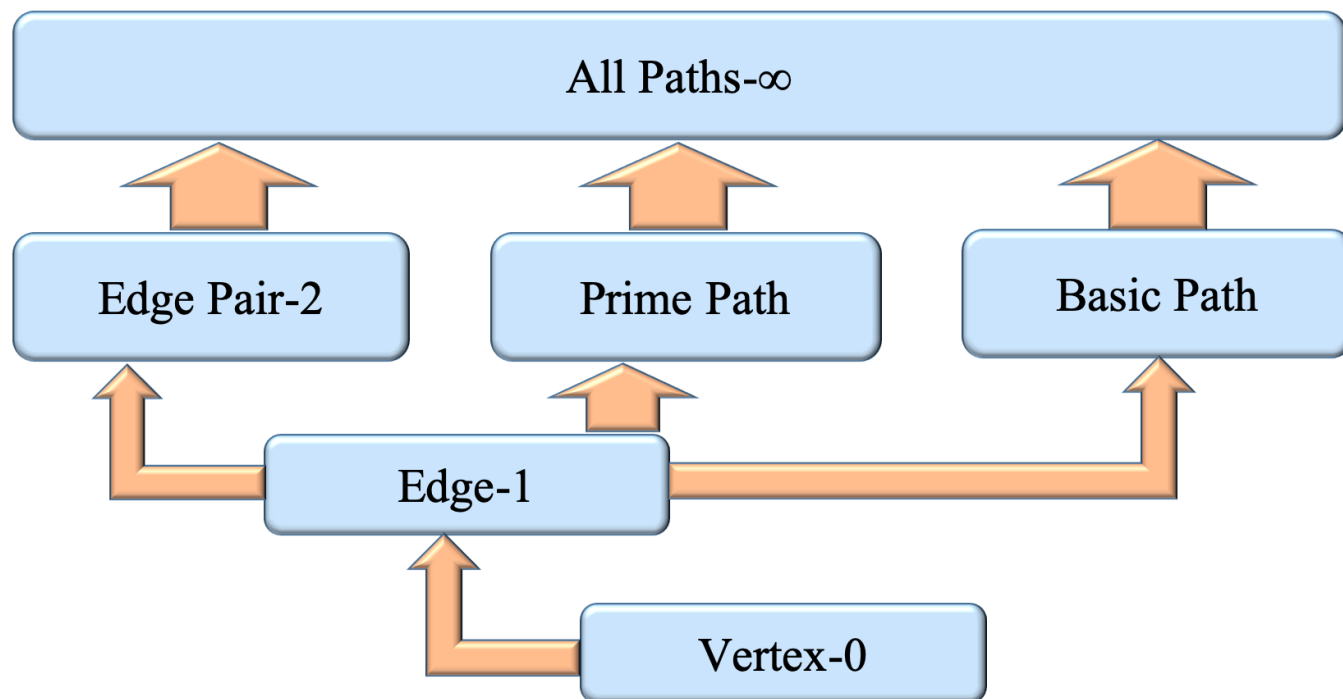
Dropout



删除少部分元素（代码）难以影响输出结果且反而可能提高了系统泛化能力

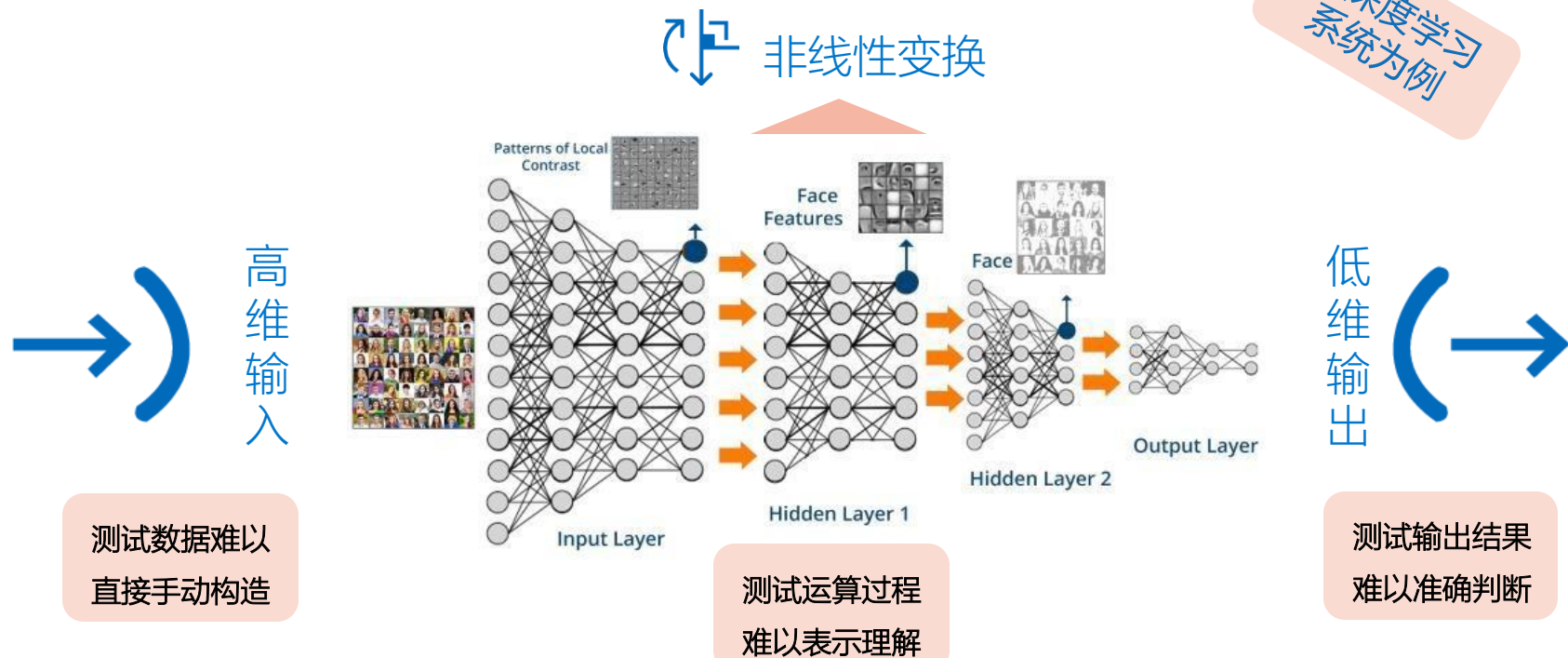


结构化覆盖测试准则



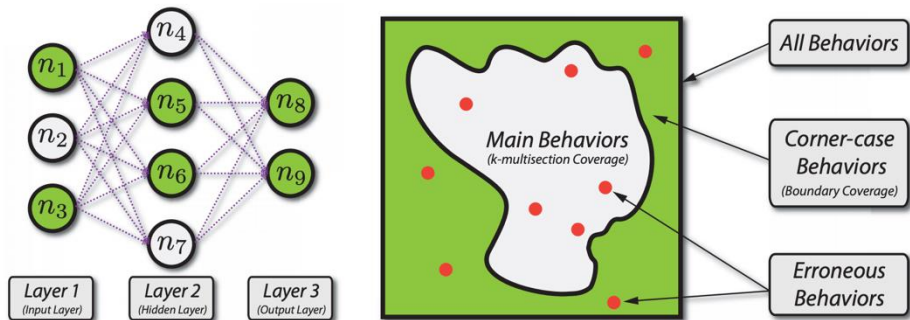


面向智能软件的 测试准则



深度学习系统：数据驱动构建的弱控制流全变量非线性变换大规模数值计算程序

神经元覆盖



对于功能区测试:

NAC: 神经元覆盖率

KMNC: 多节神经元覆盖率

$$\text{KMNCov}(T, k) = \frac{\sum_{n \in N} |\{S_i^n \mid \exists \mathbf{x} \in T : \phi(\mathbf{x}, n) \in S_i^n\}|}{k \times |N|}$$

对于边界区测试:

NBC: 神经元边界覆盖率

SNAC: 强神经元覆盖率

$$\text{NBCov}(T) = \frac{|\text{UpperCornerNeuron}| + |\text{LowerCornerNeuron}|}{2 \times |N|}$$

对于层级测试

topk神经元覆盖率

topk神经元模式

$$\text{SNACov}(T) = \frac{|\text{UpperCornerNeuron}|}{|N|}$$

$$\text{TKNCov}(T, k) = \frac{|\bigcup_{\mathbf{x} \in T} (\bigcup_{1 \leq i \leq l} \text{top}_k(\mathbf{x}, i))|}{|N|}$$

Lei Ma etc. DeepGauge: Multi-Granularity
Testing Criteria for Deep Learning Systems

$$\text{TKNPat}(T, k) = |\{(\text{top}_k(\mathbf{x}, 1), \dots, \text{top}_k(\mathbf{x}, l)) \mid \mathbf{x} \in T\}|$$



等价类测试



To find x, x' in the same class

$$s.t. S(x) = P(x)$$

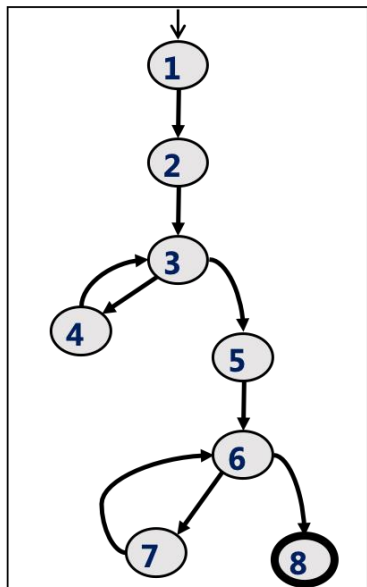
$$\text{and } S(x') \neq P(x')$$



等价类测试

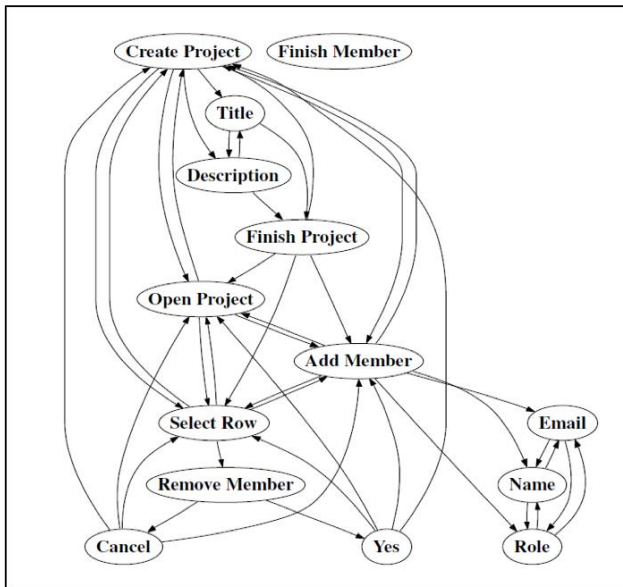


控制流程图



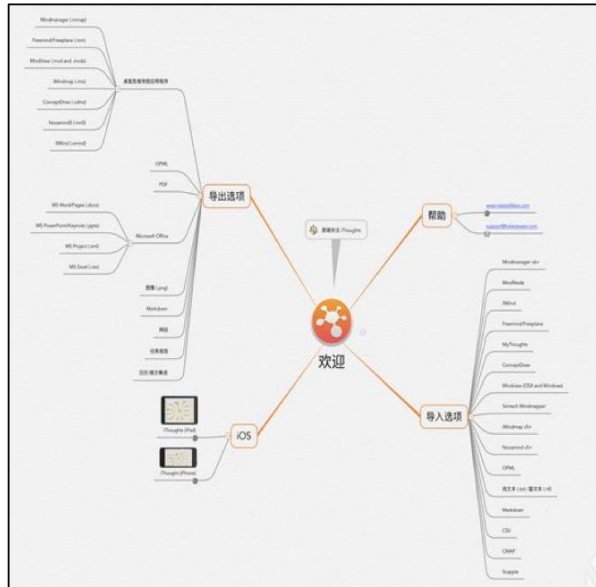
软件行为的等价类

事件流程图



用户交互的等价类

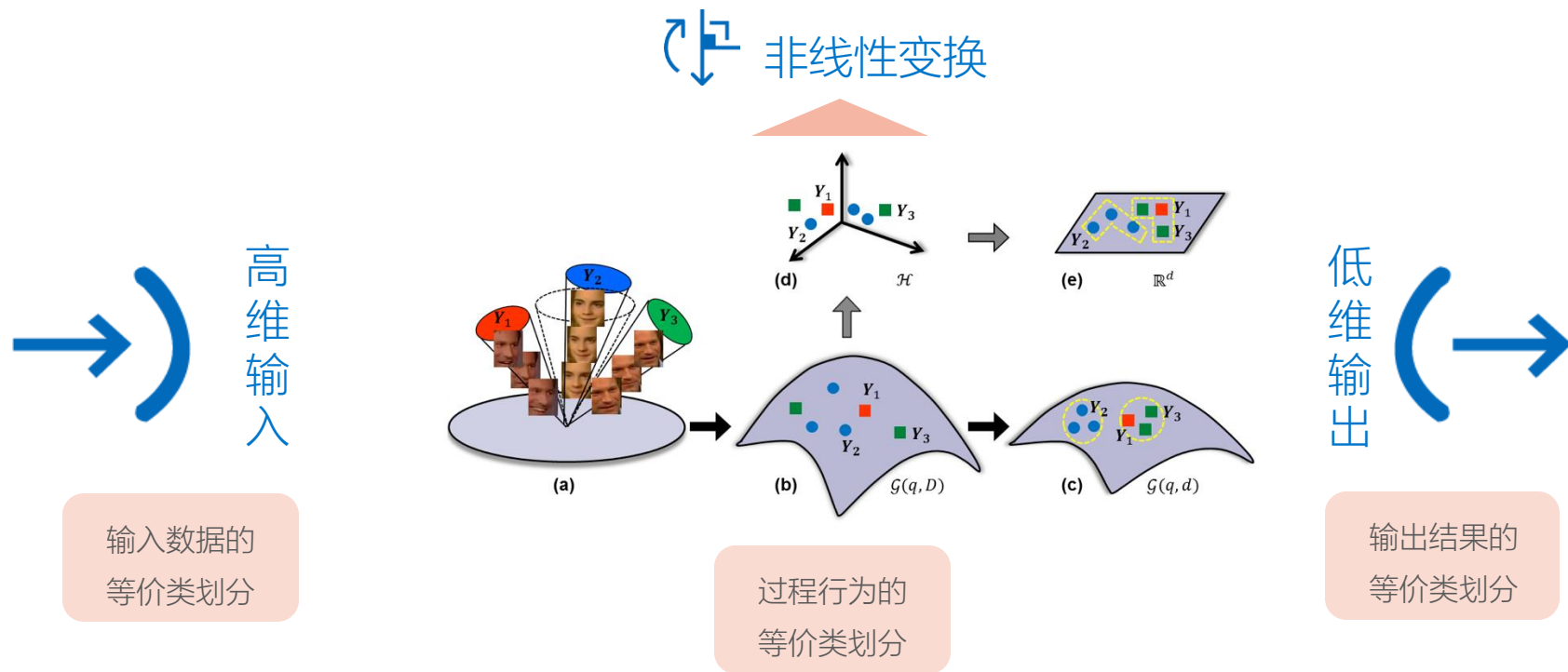
功能思维导图



功能设计的等价类



深度学习系统的 等价类测试



重新定义等价类测试：从输入接口类型或功能点等价类划分，到流形分布特征的划分



边界值侧测试



To find $x, \Delta x$,

$$s.t. \ S(x) = P(x)$$

$$\text{and } S(x + \Delta x) \neq P(x + \Delta x)$$



边界值测试



白盒：过程行为的边界

黑盒：输入值域的边界

```
public static void CSta (int [ ] numbers)
{
    int length = numbers.length;
    double var, mean, sum, varsum;

    sum = 0.0;
    for (int i = 0; i < length; i++)
    {
        sum += numbers [ i ];
    }
    mean = sum / (double) length;

    varsum = 0.0;
    for (int i = 0; i < length; i++)
    {
        varsum = varsum + ((numbers [i] - mean) * (numbers [i] - mean));
    }
    var = varsum / ( length - 1.0 );

    System.out.println ("length:      " + length);
    System.out.println ("mean:      " + mean);
    System.out.println ("variance:  " + var);
}
```

中国铁路12306
12306 CHINA RAILWAY

搜索车票/餐饮/常旅客/相关规定

首页 车票 团购服务 会员服务

车票 单程 往返 接续换乘 退改签

出发地 到达地 出发日期 2018-07-21

学生 高铁/动车

查询



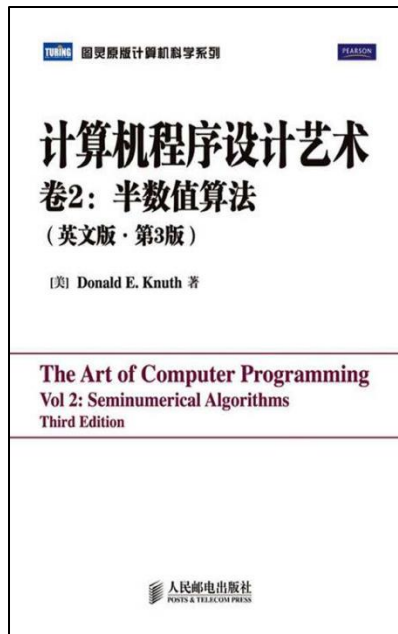
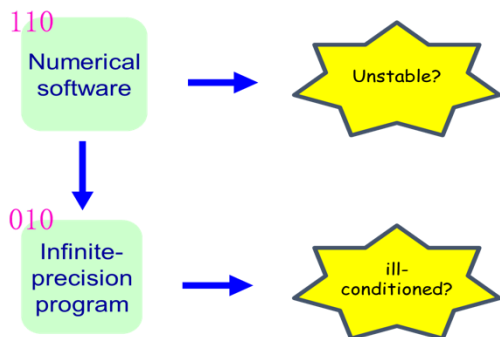
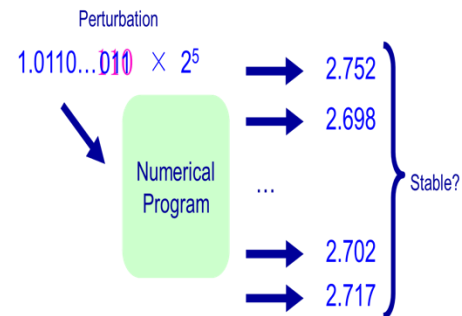
开发人员视角的边界值分析



终端用户视角的边界值分析



边界值测试： 机器视角



$$S^2 = \frac{\sum_{i=1}^n (X_i - \bar{X})^2}{n - 1}$$

输入数据的微小扰动对浮点运算数值模型带来的边界影响分析

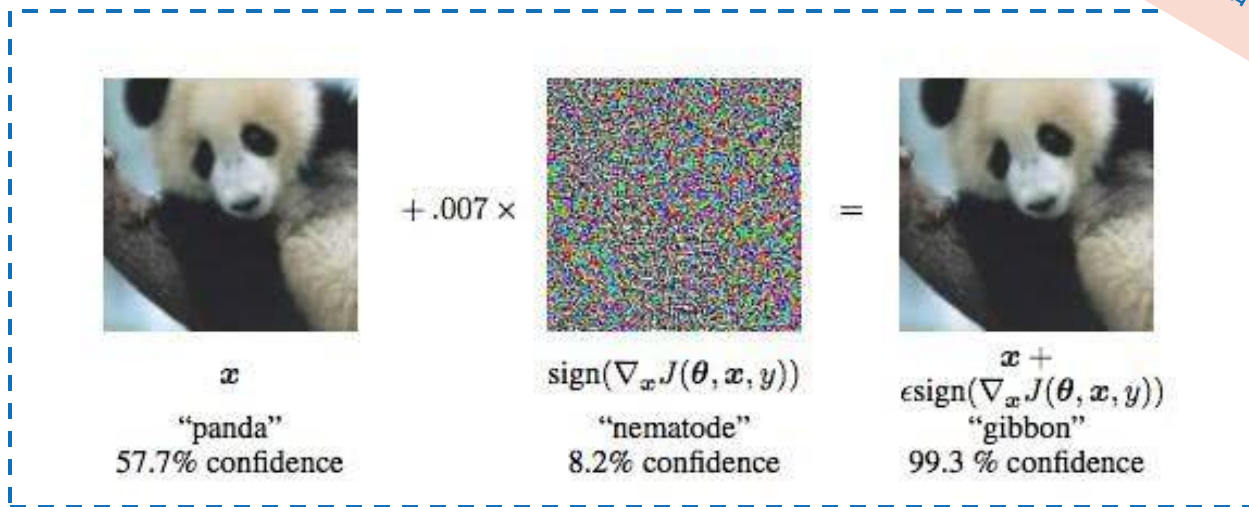
Enyi Tang, et. al. Software numerical Instability detection and diagnosis by combining stochastic and infinite-precision testing, TSE 2017.



深度学习系统的 边界值测试



机器视角



高维输入的微小扰动对非线性变换函数维度扭曲的边界影响分析

Explaining and Harnessing Adversarial Examples. Computer Science, 2014.



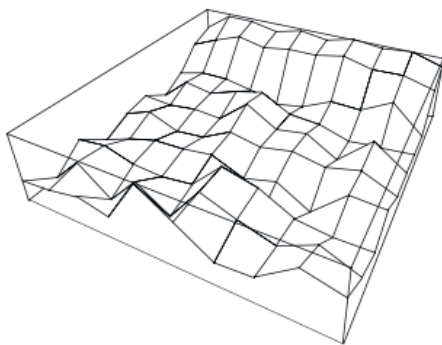
深度学习系统的边界值测试



Theorem 6 (Guarantee of the global minimum of grid search) Assume a neural network $f(x) : [0, 1]^n \rightarrow \mathbb{R}^m$ is Lipschitz continuous w.r.t. a norm metric $\| * \|_D$ and its Lipschitz constant is K . By recursively sampling $\Delta = 1/\epsilon$ in each dimension, denoted as $\mathcal{X} = \{x_1, \dots, x_{\Delta^n}\}$, the following relation holds:

$$\|f_{opt}(x^*) - \min_{x \in \mathcal{X}} f(x)\|_D \leq K \left\| \frac{\epsilon}{2} \mathbf{I}_n \right\|_D$$

where $f_{opt}(x^*)$ represents the global minimum value, $\min_{x \in \mathcal{X}} f(x)$ denotes the minimum value returned by grid search, and $\mathbf{I}_n \in \mathbb{R}^{n \times n}$ is an all-ones matrix.



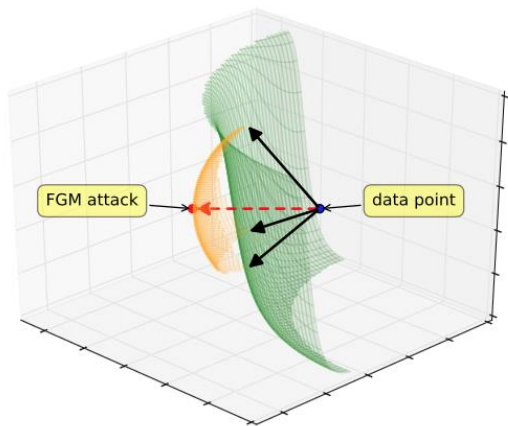
在假设神经网络 f 具有Lipschitz连续性的前提下，保证神经网络 f 的全局最小值和grid search找到最小值，由Lipschitz常数 K 控制住，为可达性分析的寻找提供误差上界。



深度学习系统的 边界值测试理论分析



Lemma 1. Given $\mathbf{g} \in \mathbb{R}^d$ and $\alpha \in [0, 1]$. The maximum number k of orthogonal vectors $\mathbf{r}_1, \mathbf{r}_2, \dots, \mathbf{r}_k \in \mathbb{R}^d$ satisfying $\|\mathbf{r}_i\|_2 \leq 1$ and $\mathbf{g}^\top \mathbf{r}_i \geq \alpha \cdot \|\mathbf{g}\|_2$ is $k = \min \left\{ \left\lfloor \frac{1}{\alpha^2} \right\rfloor, d \right\}$.



对抗样本所在空间的剖面估计：对每一个确定的数据点，估计这个数据点在神经网络 f 上对抗样本所在空间的剖面。



面向智能软件的 群体智能测试



Error Generation

综合利用众包手段和及其手段来进行对抗数据的生成

Error Validation

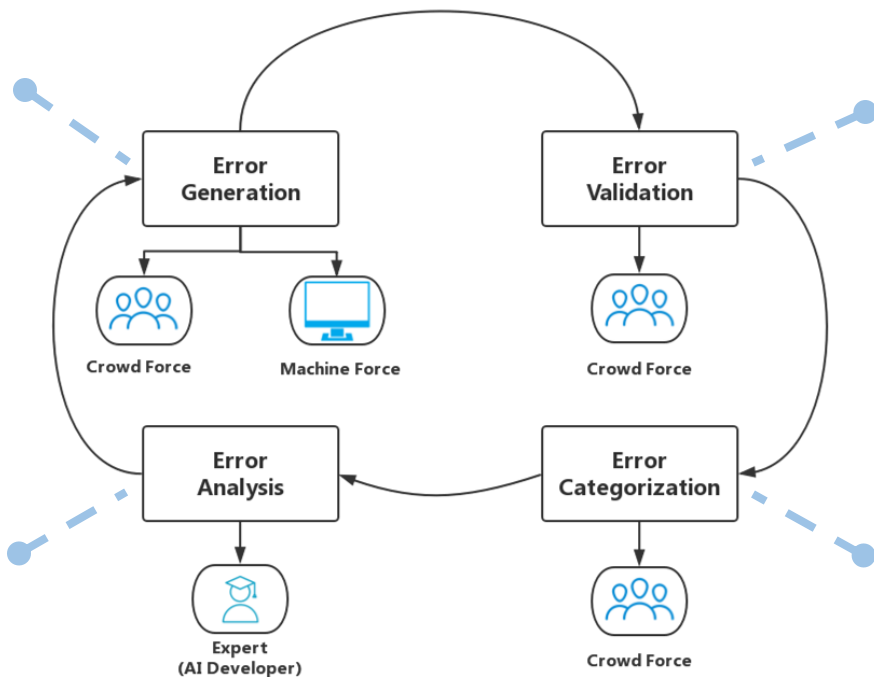
利用群智手段对第一阶段的样本进行清洗，保留高质量的错误测试样本

Error Analysis

为AI开发者提供良好的错误分析手段，多种可视化方式，便于定位

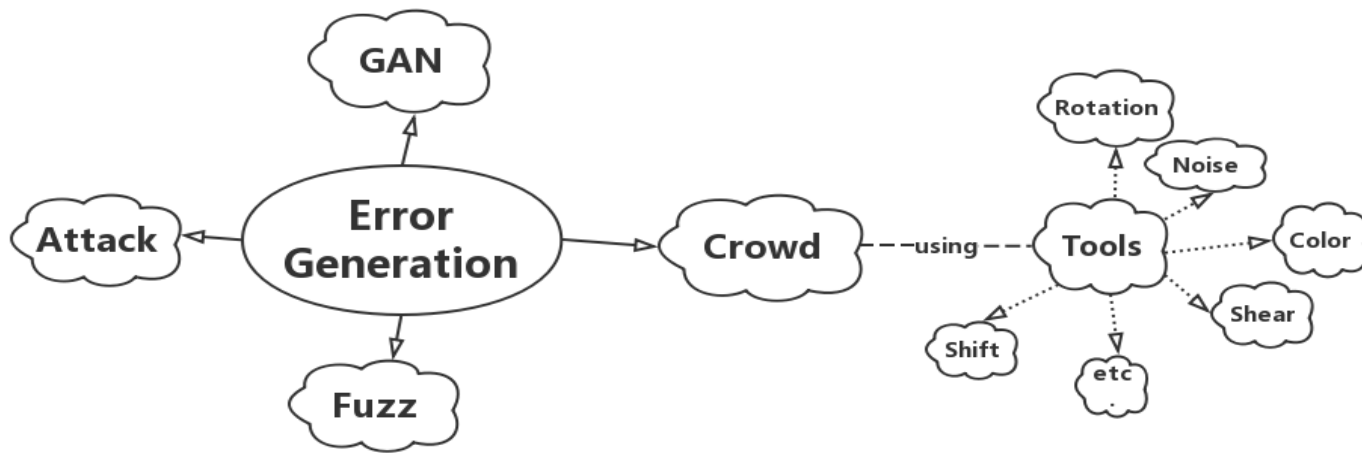
Error Categorization

对上阶段的错误样本进行不同维度的分类，为错误分析做准备





面向智能软件的 群体智能测试



所示为 机器手段，包含GAN、对抗技术、Fuzz技术等方式

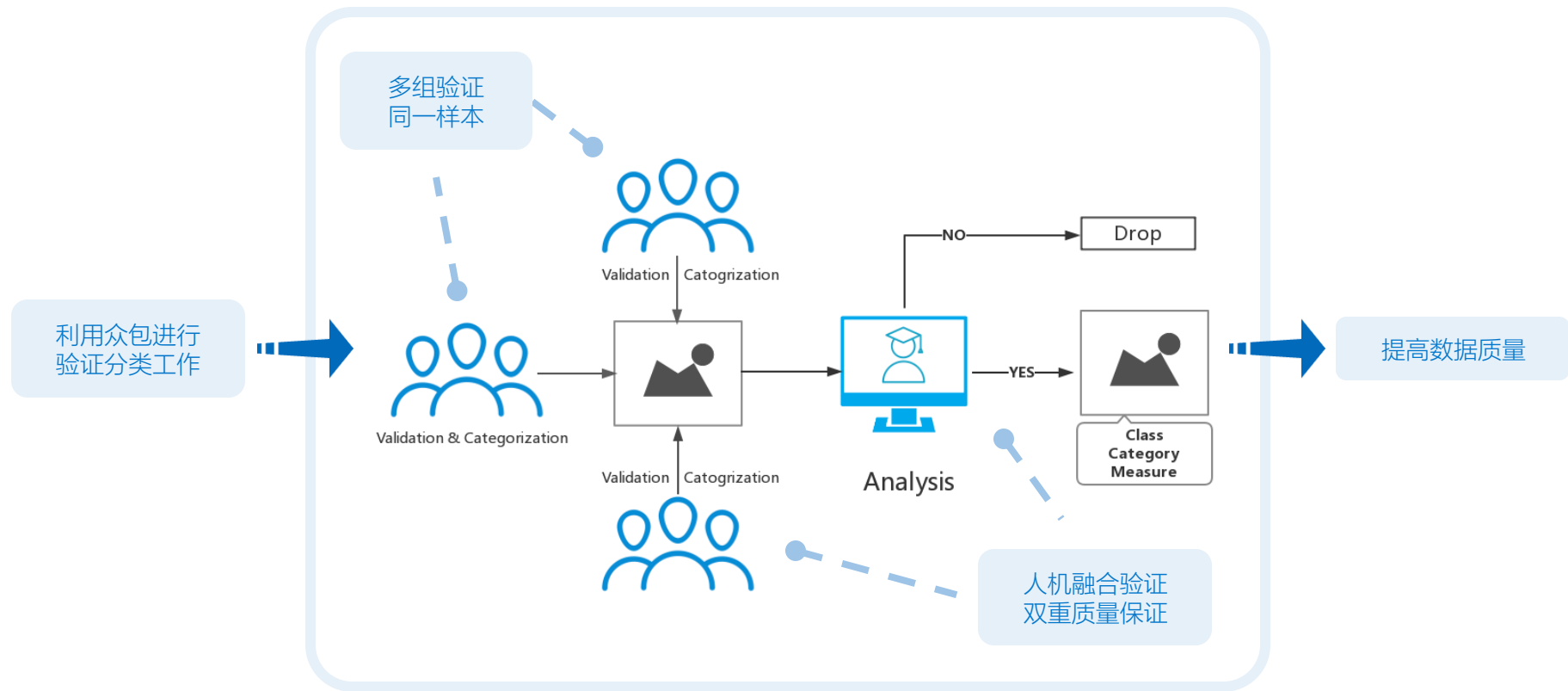


所示为 众包手段，包含画笔，旋转，噪音，平移，变色等辅助工具

如何构建高维测试数据是
首个挑战



面向智能软件的 群体智能测试





面向智能软件的 群体智能测试



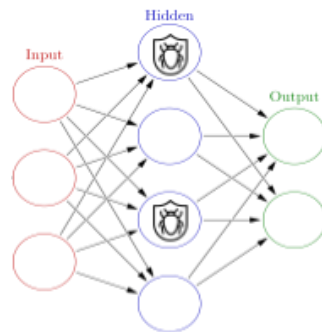
可视化理解



缺陷理解



结构参数调节



进一步，可将优化后的模型继续投入到Error Generation阶段进行迭代，直至模型优化达到预期



电磁信号蜕变

保持电磁数据的场景性

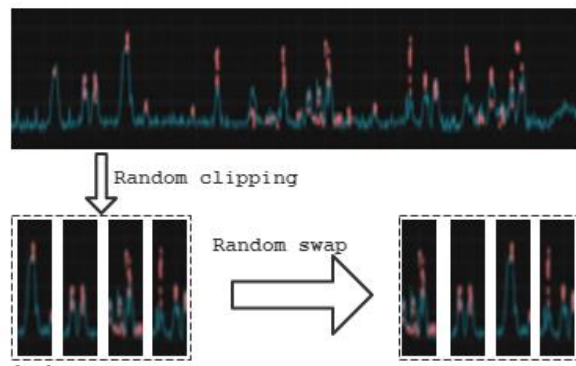
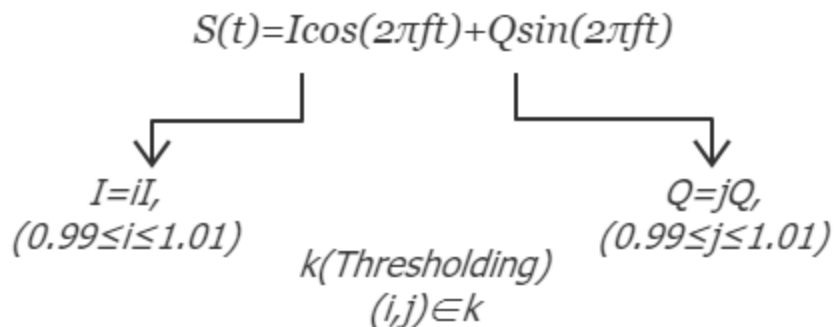
电磁信号在传播过程会受到不同程度的外界因素的干扰，收集到的电磁数据无法像图片数据一样能够保持场景性。

$$S(t) = I \cos(2\pi ft) + Q \sin(2\pi ft)$$

等价蜕变：由于无法从电磁信号本身对电磁数据进行规律分析，因此选择从I通道数据与Q通道数据上进行等价处理。

关系一：对I/Q数据进行放缩处理，模拟现实场景中电磁波折射的现象。

关系二：交换连续I/Q数据段，模拟信号的非规律性关联。



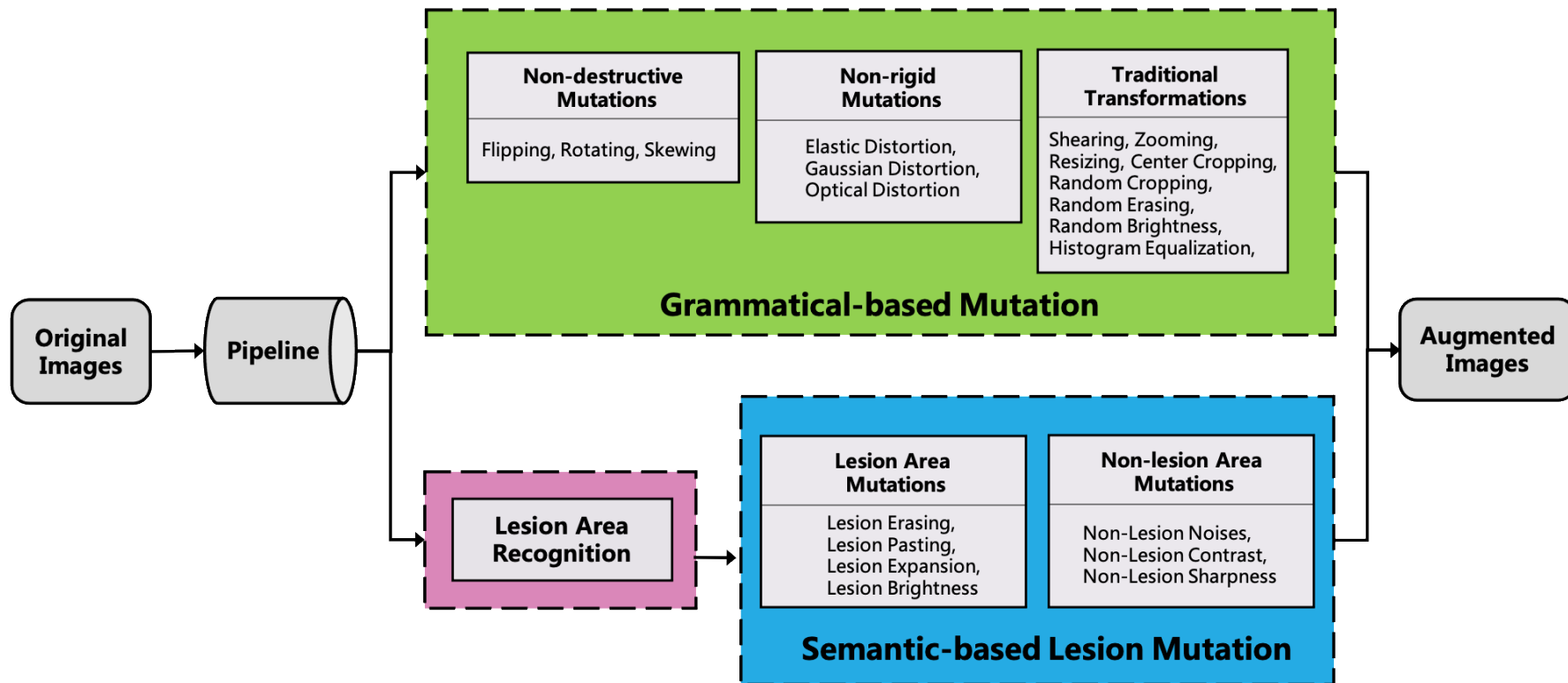




表 4.6: 图像特征变异扩增方法列表

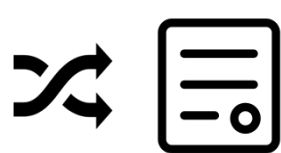
扩增方法	方法介绍	方法原理
水平翻转	沿图像水平方向翻转	翻转后图像的每个像素的横坐标相对于图像的垂直中心线对称于翻转前相应像素的横坐标
垂直翻转	沿图像垂直方向翻转	翻转后图像的每个像素的横坐标相对于垂直翻转中图像的水平中心线对称于翻转前相应像素的横坐标
随机旋转	将图像旋转任意角度	以旋转中心为原点, 将图像旋转角度 θ
倾斜变换	将图像向任意方向倾斜	在水平和垂直方向上将单元内的每个点扭曲一定的角度
弹性变换	在保持图像的纵横比下对图像进行弹性变换	由网格宽度和高度控制扭曲的精细程度
高斯畸变	在保持图像的纵横比下对图像进行高斯畸变	使用高斯滤波过滤像素每个维度的偏差矩阵, 并用新像素值替代原始像素位置的值得
光学失真	在保持图像的纵横比下对图像进行光学失真	包括桶形和枕形畸变
随机裁剪	根据要获取的区域百分比裁剪图像的随机区域	在图像的随机区域进行裁剪
错切变换	将图像错切指定的度数	以某一系数对单方向的像素点进行变换
随机擦除	对图像任意区域进行随机擦除	通过用随机像素值替换任意位置的矩形区域
缩放变换	对图像大小进行调整	对图像进行放大或缩小, 同时保持其原尺寸大小
直方图均衡化	利用图像直方图调整图像对比度	把原始图的直方图变换为均匀分布的形式
随机亮度	对图像亮度进行调整	随机改变图像亮度

表 4.7: 领域语义变异扩增方法列表

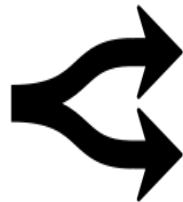
扩增方法	方法介绍	方法原理
病灶擦除	对图像中的病灶区域进行擦除	使用随机像素值替换病灶区域, 以创建没有病灶区域的新图像
病灶亮度	随机改变图像病灶区域亮度	将病灶区域随机变亮或变暗, 以模拟医生实际读片时光线对病灶区域的影响
病灶复制	对图像中的病灶区域进行复制	增加医学图像中病灶的数目, 模拟疾病在不同个体或发展阶段的多样性
病灶扩张	扩张图像中的病灶区域	扩大病变区域, 即扩大病变范围, 纵向模拟疾病随时间的发展
病灶移动	移动图像中的病灶区域	将提取的病灶区域移动到图像中的随机位置, 模拟病灶的活动性或个体表现差异
噪声处理	在图像的非病灶区域进行噪声处理	在图像中非病灶区域随机添加噪声。噪声根据图像的信噪比, 随机生成一些图像内部的像素位置, 并且对这些像素点赋值
对比度处理	改变非病灶区域对比度	在图像中的非病灶区域改变对比度。在医疗影像中, 对比度会对实际读片效果产生影响
锐度处理	改变非病灶区域锐度	在图像中的非病灶区域改变锐度。锐度由不同色调或颜色区域之间的边界定义, 反应了图像细节数量的多少



面向深度学习的 测试多样性度量



随机抽取测试集



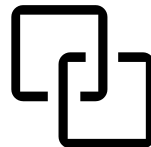
神经网络编码



神经网络预测



熵度量



多样性分析



AUC统计



01 基于输出层对于各个分类的概率分析计算

以Gini系数为基础

$$\xi(t) = 1 - \sum_{i=1}^N p_{t,i}^2$$

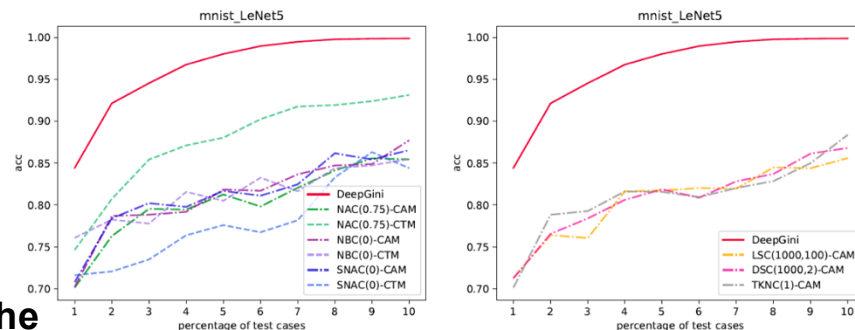
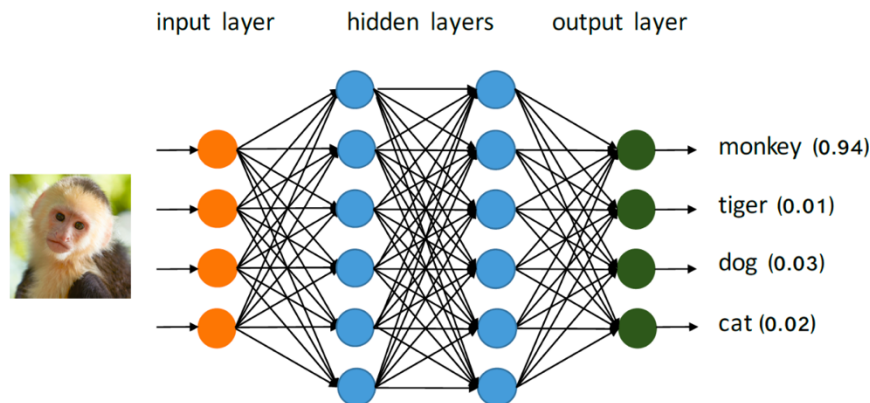
轻量，具有极好计算效率

02 应用极其广泛

能够快速选取具有较好测试效果的用例

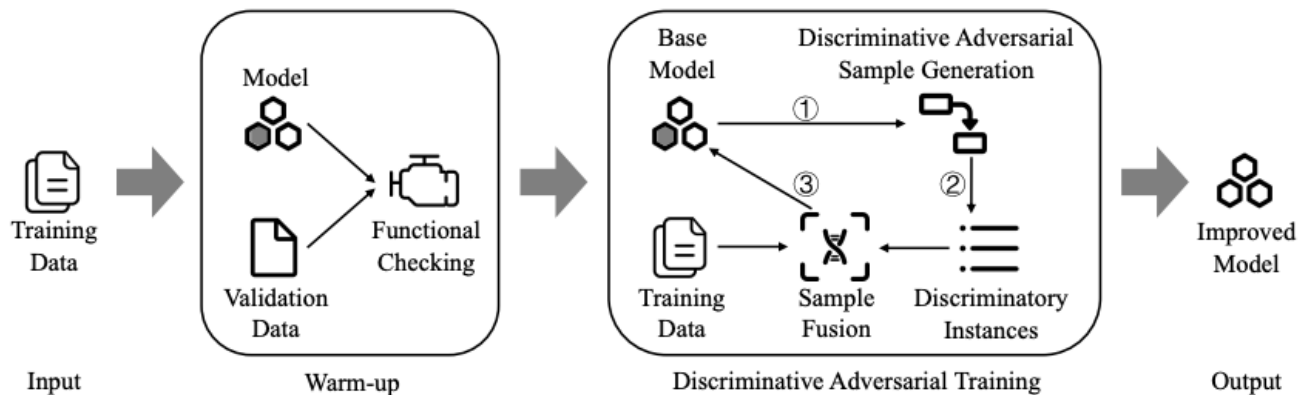
选取的测试用例用于重训练能够显著提升模型精度

DeepGini: prioritizing massive tests to enhance the robustness of deep neural networks. ISSTA 2020





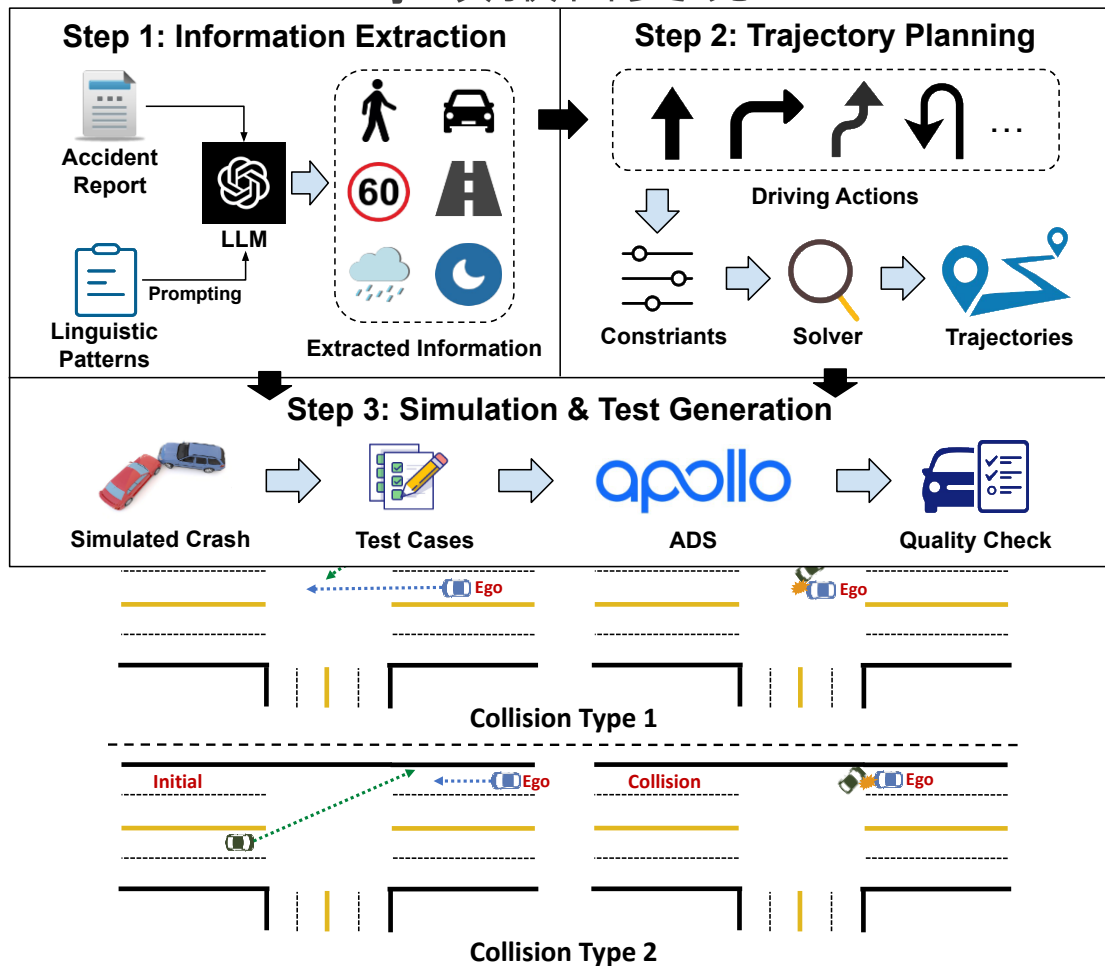
RULER



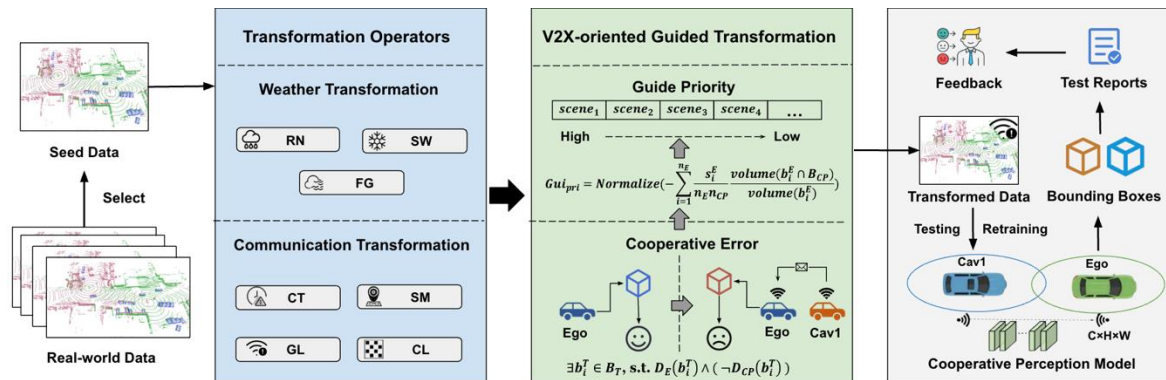
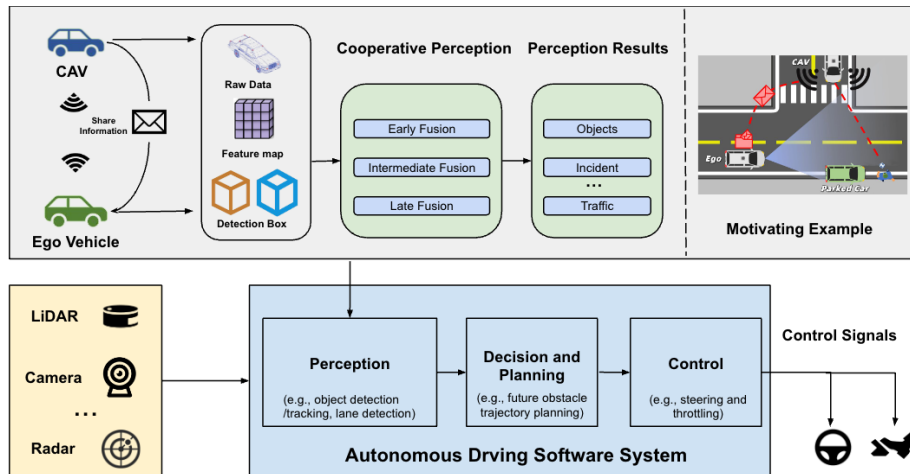
雪崩时,没有一片雪花是无辜的

RULER: Discriminative and Iterative Adversarial Training for Deep Neural Network Fairness, ESEC/FSE 2022

事故报告复现



协同感知系统测试





深度学习框架



MindSpore



HUAWEI

AI应用

HiAI Service	General APIs	Advanced APIs	Pre-integrated Solutions
HiAI Engine	ModelArts		

应用技能

MindSpore	TensorFlow	PyTorch	PaddlePaddle	—
-----------	------------	---------	--------------	---

框架

CANN
(Compute Architecture for Neural Networks)

芯片使能

Ascend-Nano	Ascend-Tiny	Ascend-Lite	Ascend	Ascend-Mini	Ascend-Max
-------------	-------------	-------------	--------	-------------	------------

IP和芯片

消费终端

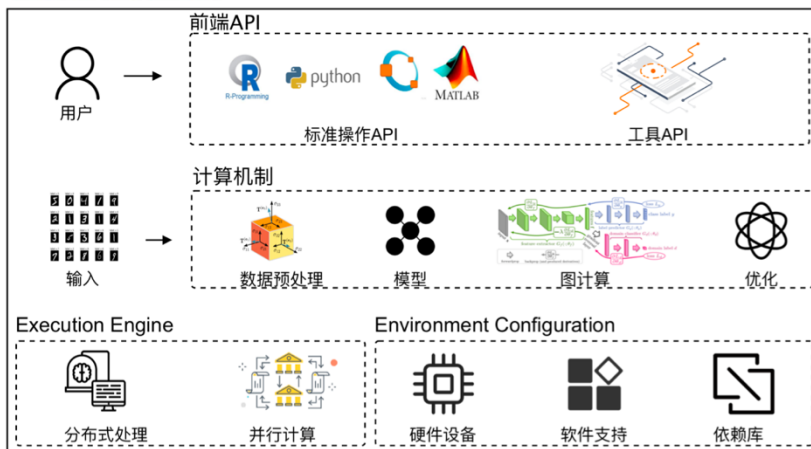
公有云

私有云

边缘计算

IOT行业终端

全场景





深度学习框架



DL model
e.g., AlexNet LeNet VGG

Deep Neural Network

Libraries



TensorFlow



Caffe



PyTorch

...



MNN

interface

Operators

Conv

Pooling

Norm

ReLU

...

Tanh

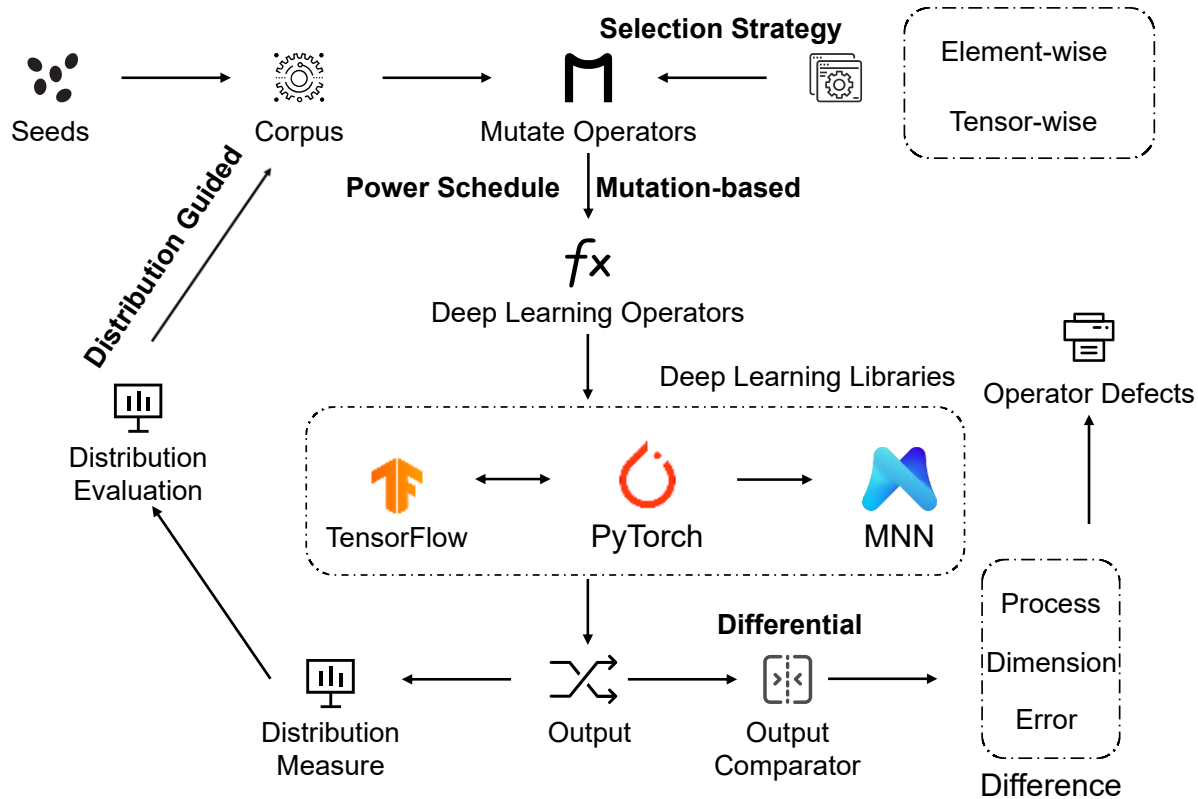
Hardware

CPU

GPU



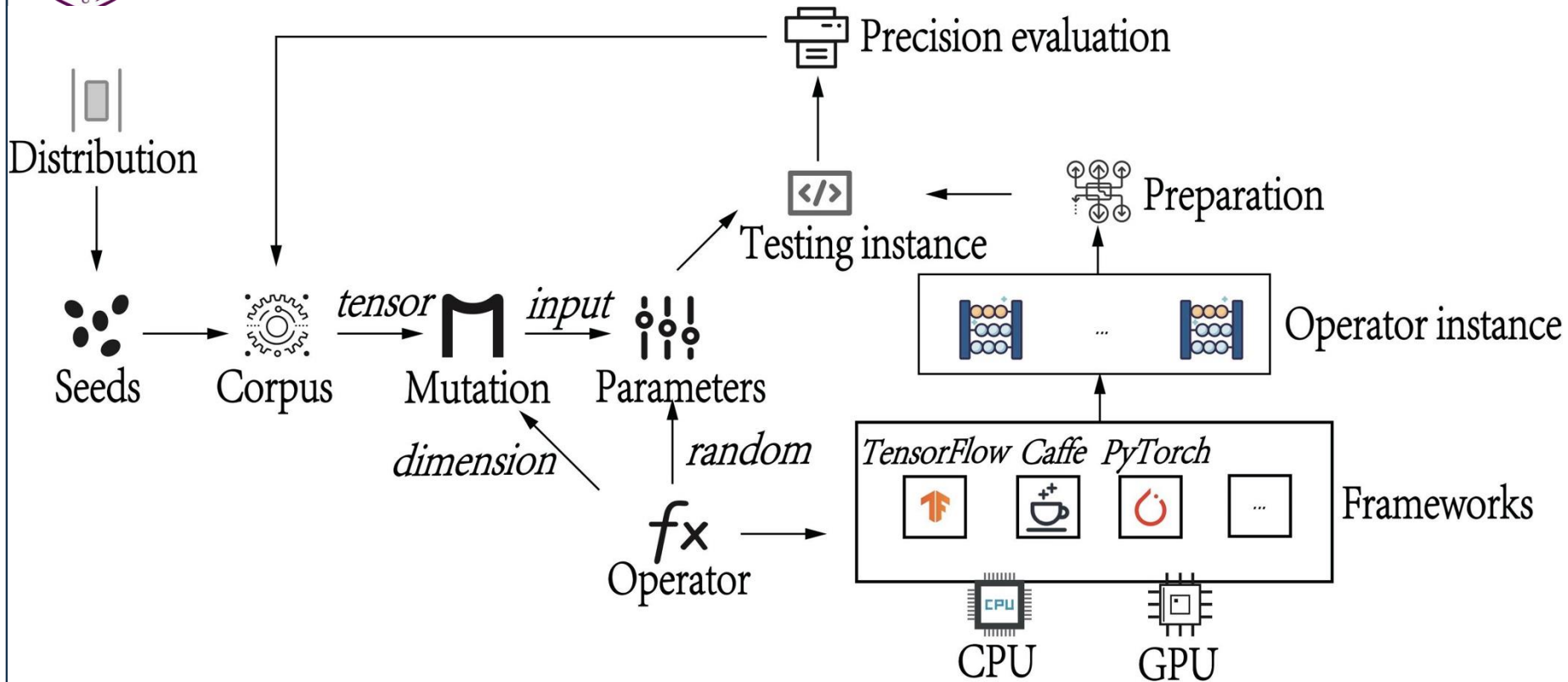
Duo



Duo: Differential Fuzzing for Deep Learning Operators, TR 2021

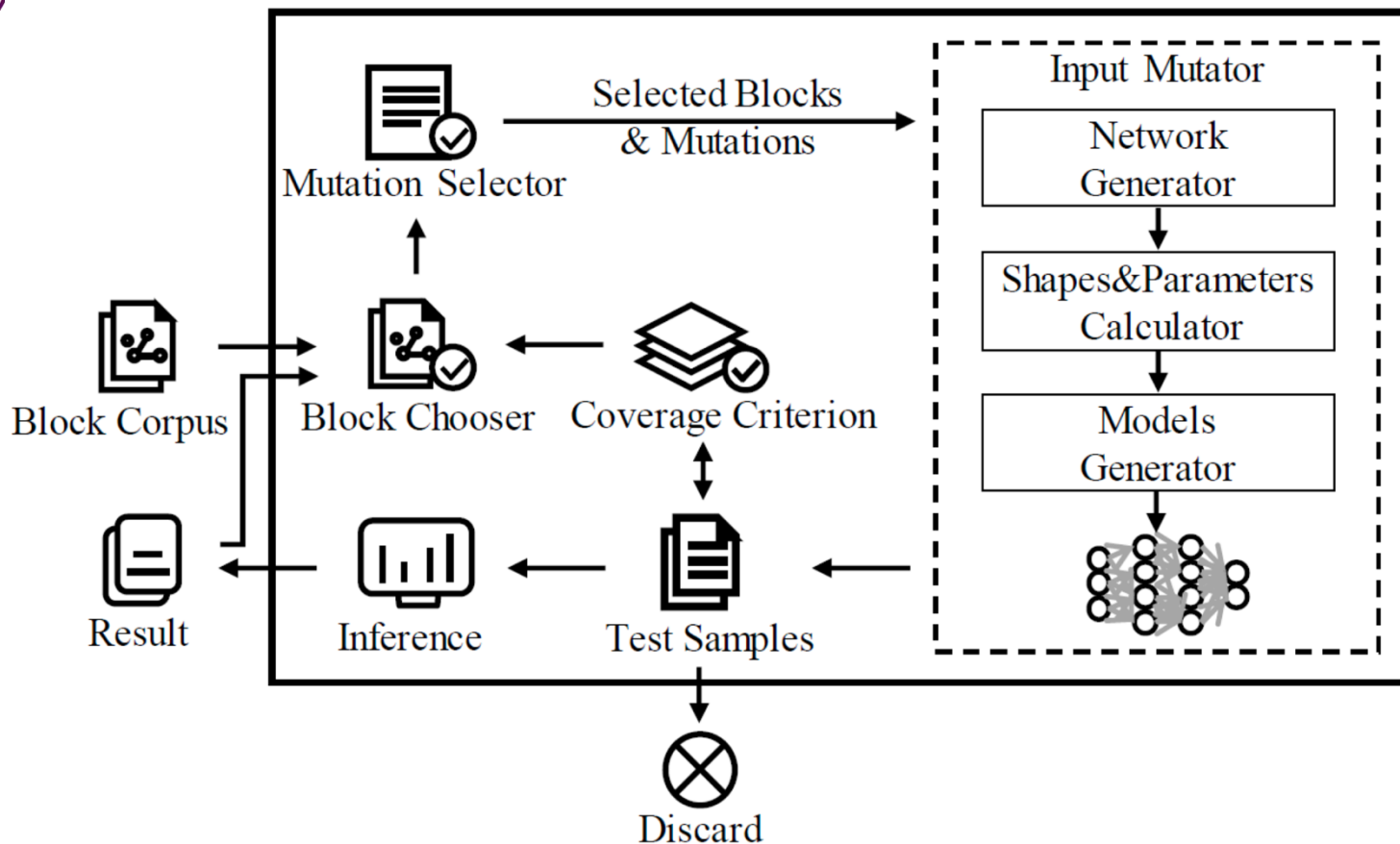


Predoo



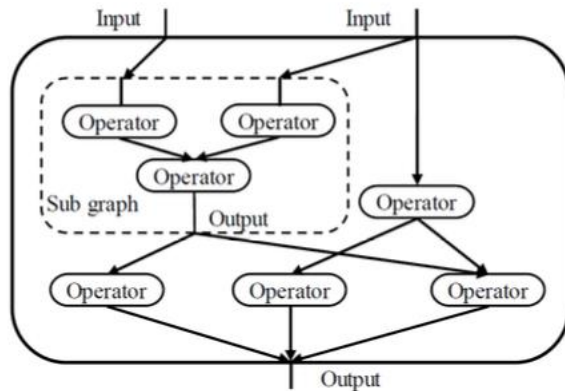


GraphFuzz



Graph-based Fuzz Testing for Deep Learning Inference Engine, ICSE 2021

GraphFuzz



Operator Type Coverage(OTC)

$$OTC(I) = \frac{\sum OTC_op(c, I)}{n_t}$$

Input Degree Coverage(IDC)

$$IDC(I) = \frac{\sum IDC_op(c, I)}{n_t}$$

Single Edge Coverage (SEC)

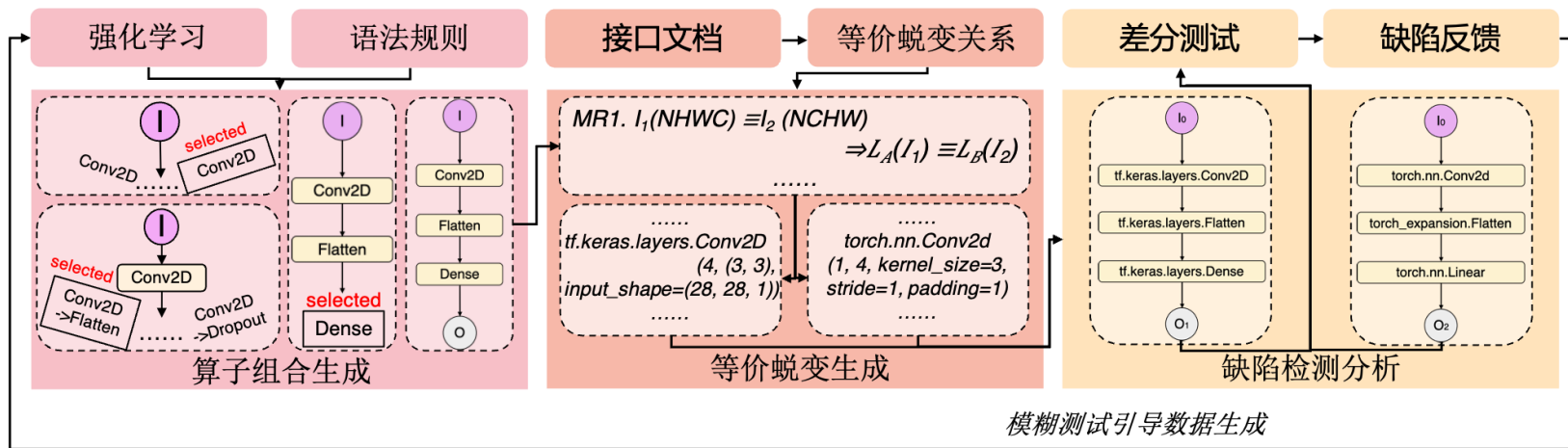
$$SEC(I) = \frac{\sum SEC_op(c, I)}{n_t}$$

Shapes&Parameters Coverage(SPC)

$$SPC(I) = \frac{\sum SPC_op(c, I)}{n_t}$$

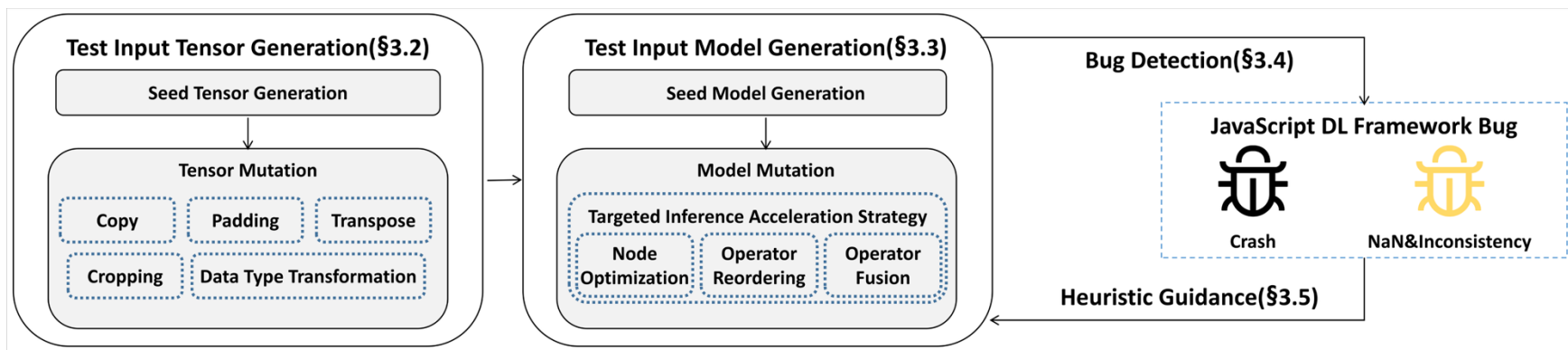


基于语法生成的差分模糊测试



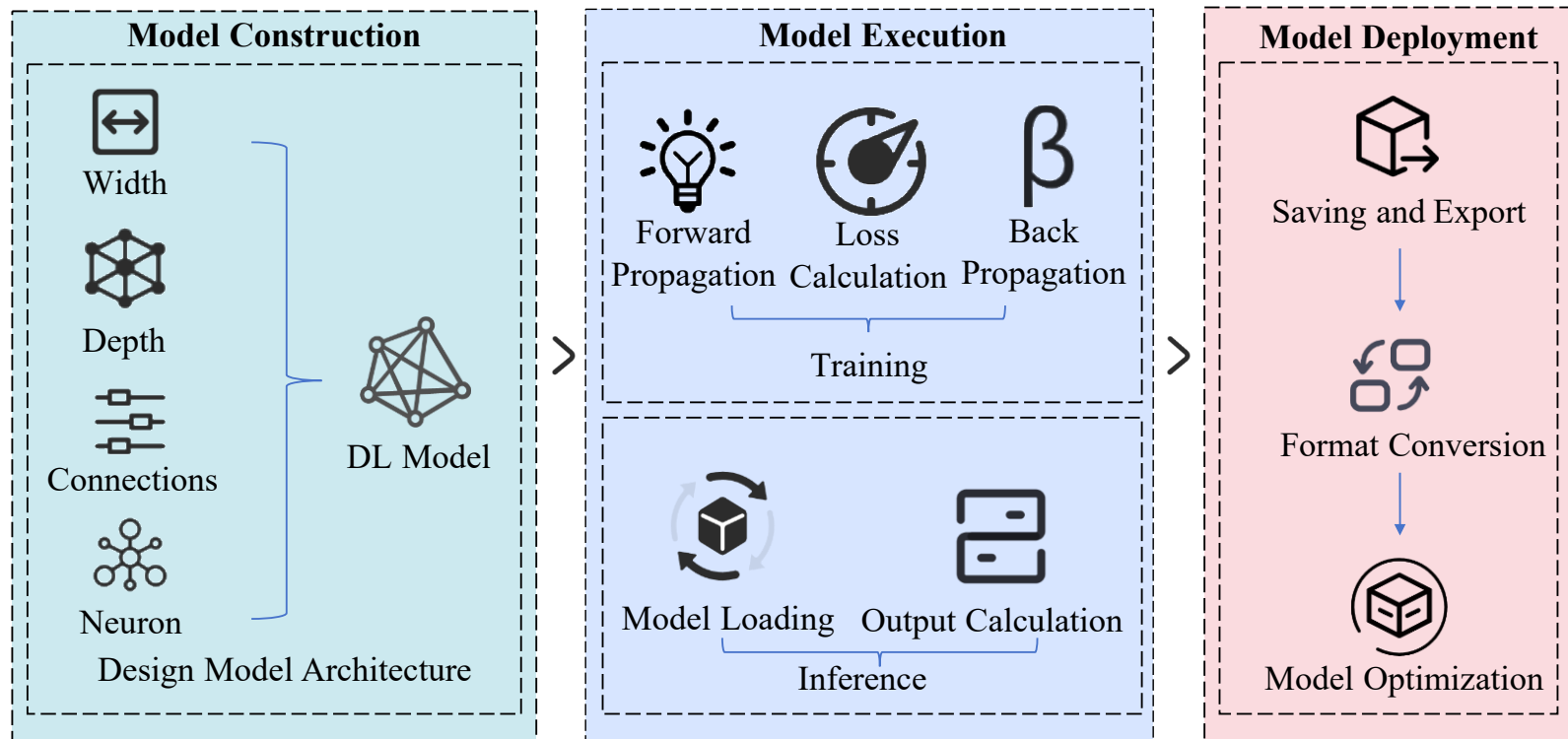


——基于变异的JavaScript深度学习框架测试——





——基于开发者经验的深度学习框架测试——



DevMuT: Testing Deep Learning Framework via Developer Expertise-Based Mutation, ASE 2024



zychen@nju.edu.cn
fangchunrong@nju.edu.cn

Thank you!